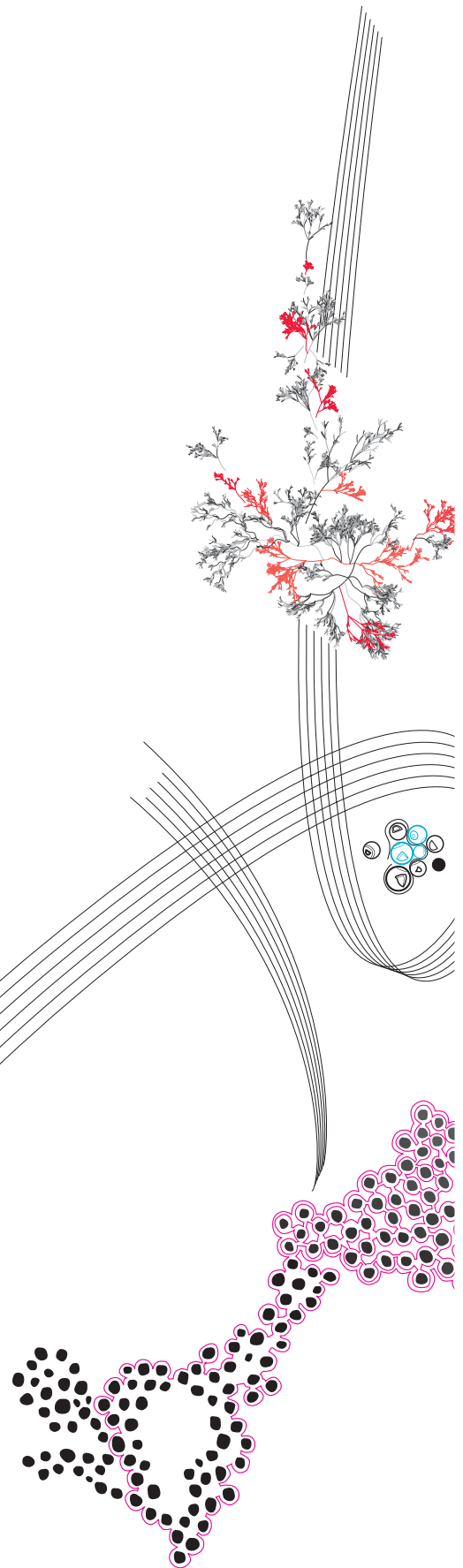




MSc Computer Science
Final Project

A Hierarchical Time-Aware Transformer for Flow-Level Network Intrusion Detection

Xiaoyan Xiong

Supervisor: Le Viet Duc

August, 2026

Department of Computer Science
Faculty of Electrical Engineering,
Mathematics and Computer Science,
University of Twente

Contents

1	Introduction	1
2	Related Work	3
2.1	Signature-Based Detection and Flow-Statistics Models	3
2.2	Deep Learning for Packet and Flow Sequences	3
2.3	Transformer-Based Traffic Representation	4
2.4	Self-Supervised and Encrypted-Traffic Representation Learning	4
2.5	Temporal, Relational, and Cross-Flow Context	5
2.6	Research Gap	6
3	Dataset Construction and Preprocessing	8
3.1	Data Provenance and Scope	8
3.2	External Evaluation Datasets	8
3.3	PCAP-to-CSV Extraction	9
3.4	Alert-Derived Label Construction	10
3.5	Raw Dataset Composition	10
3.6	Raw Packet and Flow Schemas	11
3.7	Integrity Checks and Packet-Flow Alignment	12
3.8	Feature Selection and Leakage Control	13
3.9	Train-Fitted Feature Transformations	14
3.10	Packet-Sequence Construction	14
3.11	Chronological Train-Validation-Test Partition	15
3.12	Stage 1 Artifacts Supplied to Stage 2	15
3.13	Validity, Privacy, and Reproducibility Considerations	16
4	Proposed Methodology	17
4.1	Architectural Overview	17
4.2	Stage 1: Intra-Flow Time-Aware Representation	19
4.2.1	Packet-Record Projection	19
4.2.2	Cumulative Time-Aware Encoding	19
4.2.3	Flow-Statistics Encoder	20
4.2.4	Bounded Flow-Conditioned Token Modulation	20
4.2.5	Masked Transformer Encoder	20
4.2.6	Hybrid Mask-Aware Pooling	21
4.2.7	Stage 1 Fusion Variants	22
4.3	Stage 2: Start-Time-Ordered Inter-Flow Modelling	22
4.3.1	Embedding Interface and Chronological Order	22
4.3.2	Start-Time-Ordered Relation-Specific Contexts	22
4.3.3	Context-Age Positional Encoding	24
4.3.4	Target-Query Gated Multi-Head Attention	24
4.4	Imbalance-Aware Objective and Stage Interface	25
4.4.1	Focal Objective and Controlled Sampling	25
4.4.2	Sequential Stage Interface	25

5	Experimental Methodology	26
5.1	Evaluation Manifests and Leakage Controls	26
5.1.1	External Evaluation Protocol	26
5.2	Experimental Matrix	27
5.2.1	Stage 1 Baselines	27
5.2.2	Stage 1 Ablations and Sensitivity Tests	28
5.2.3	Stage 2 Baselines and Context Ablations	28
5.3	Training, Model Selection, and Thresholding	28
5.3.1	Principal Training Configuration	28
5.3.2	Checkpoint and Decision-Threshold Selection	29
5.4	Evaluation Metrics	29
5.4.1	Class-Wise and Macro-Averaged Measures	30
5.4.2	False-Positive Rate and Operational Error Counts	30
5.4.3	Threshold-Independent Ranking Measures	30
5.5	Efficiency and Resource Measurement	31
6	Results and Discussion	32
6.1	Stage 1: Intra-Flow Representation	32
6.1.1	Architecture and Temporal-Encoding Effects	32
6.1.2	Packet and Flow-Statistics Integration	34
6.1.3	Sequence Length and Packet Selection	35
6.2	Stage 2: Inter-Flow Context	36
6.2.1	Gain over Independent Flow Classification	36
6.2.2	Which Relation Defines Useful Context?	37
6.2.3	How Much Context Is Useful?	38
6.2.4	Stage 2 Architecture Comparison	38
6.2.5	Efficiency and Operational Trade-Offs	39
6.3	External Generalisation	40
6.4	Cross-Stage Discussion	41
7	Conclusion and Future Work	43
7.1	Conclusions	43
7.2	Limitations and Scope	43
7.3	Future Work	44
7.3.1	Resource-Efficient Self-Supervised Pre-Training	44
7.3.2	Attack-Adaptive Relation-Aware Context Modelling	44
7.3.3	Streaming Deployment, Efficiency, and Robustness	45
A	Complete Packet and Flow Field Specification	51

Abstract

Many flow-level network intrusion detection pipelines reduce each connection to flow statistics and then classify flows in isolation. This efficient abstraction suppresses packet ordering and non-uniform timing. It can also obscure evidence that becomes visible only across related connections. To retain both levels of structure, this thesis develops a hierarchical detector that does not consume payload content. A custom Suricata output module extracts aligned packet records and flow statistics from an enterprise PCAP. Stage 1 encodes the ordered packet metadata using positional and cumulative-time information. A separate flow-statistics branch conditions the packet tokens, and Stage 1 exports one fused representation per flow. Stage 2 uses the target representation to query a start-time-ordered history of earlier related flow embeddings and then produces the final prediction.

Chronological partitions are used throughout the evaluation. Because positive flows are scarce, the analysis gives priority to macro-F1, average precision (AP), positive-class performance, confusion matrices, and false-positive rate. For consistency with the experiment logs and figures, AP is labelled PR-AUC throughout. On the controlled Stage 1 flow test manifest, the complete fused representation reaches a macro-F1 of 0.8954 and a PR-AUC of 0.8763. It exceeds the flow-statistics baseline and the recurrent, convolutional, and encoding controls. On the separate Stage 2 flow manifest, source-host context raises macro-F1 from 0.8954 to 0.9275 and PR-AUC from 0.8763 to 0.9345. Both false-positive and false-negative counts decline. The target-query model also surpasses the no-context, vanilla Transformer, LSTM, GRU, and CNN-LSTM alternatives. Its model-level latency is comparable to that of the vanilla Transformer.

External evaluation uses five non-overlapping 20,000-flow windows from the public CICIDS2017 Wednesday capture and a 38,513-flow company capture obtained from a different server. Pooled macro-F1 across the five public windows is 0.8966, while the separate-server capture reaches 0.9026. Taken together, the results suggest that explicit intra-flow timing and selective inter-flow context supply complementary evidence. The external results provide preliminary evidence of transfer, although performance varies across time windows.

Keywords: Network intrusion detection, packet sequence, time-aware Transformer, flow representation, cross-flow context, class imbalance

1 Introduction

Enterprise systems, cloud platforms, mobile networks, and Internet-of-Things (IoT) deployments collectively produce large and heterogeneous traffic streams. Continuous monitoring is needed to identify activities such as reconnaissance, exploitation attempts, botnet communication, distributed denial-of-service (DDoS), and multi-stage intrusion campaigns. Network intrusion detection systems (NIDSs) consequently form an important layer of cybersecurity infrastructure [1, 2]. Their practical design nevertheless remains difficult. Contemporary attacks may be distributed, adaptive, and temporally extended, while encryption restricts direct payload inspection and increases reliance on headers, metadata, and behavioural traffic characteristics [3, 4].

Operational deployments continue to rely on signature engines such as Snort and Suricata because their rules are interpretable, computationally efficient, and effective against known threats [1, 5]. Suspicious traffic is recognised through patterns designed and maintained by domain experts. Such rules remain valuable, but their coverage is necessarily bounded: unseen attacks and evolving variants may not match an existing signature, and malicious intent may emerge only from temporal or contextual regularities spanning several flows [2, 6]. Rule-based analysis alone is therefore unlikely to provide sufficient coverage as traffic volumes and attack behaviours continue to diversify.

Learning-based NIDSs seek greater adaptability by estimating decision boundaries from observed traffic rather than depending exclusively on predefined rules. In conventional pipelines, a bidirectional flow is represented by flow statistics such as its duration, packet and byte counts, rates, and inter-arrival-time summaries [7, 8]. Compact fixed-length vectors support efficient large-scale processing, but the compression removes detail about the process that generated each flow. Packet order, bursts, directional transitions, and uneven temporal gaps can all disappear during aggregation. For attacks whose distinguishing evidence lies in how a connection unfolds rather than only in its total volume, that information loss is consequential.

Richer traffic representations have been investigated with convolutional and recurrent neural networks, LSTMs, GRUs, and autoencoder-based architectures [9, 10, 11]. These models can capture non-linear feature interactions while reducing manual feature engineering. Yet their inputs often remain fixed flow-statistics vectors. In some sequence-based studies, the sequence is formed from fixed-size flow attributes rather than the packets observed within a connection. Such formulations preserve only part of the natural hierarchy of network traffic: packets constitute flows, and related flows in turn express host- or endpoint-level behaviour.

Self-attention offers a suitable mechanism for addressing this hierarchy because distant sequence elements can interact directly and training can be parallelised [12]. Transformer-based systems have consequently achieved competitive results in traffic classification and intrusion detection [13, 14, 15]. Time-aware variants show, in particular, that detection can benefit when order and elapsed time are encoded as distinct signals [14]; FlowTransformer demonstrates the architectural flexibility available for flow-based NIDS evaluation [15]. Even so, a substantial portion of this literature continues to represent each flow as one flat item. Packet-level dynamics inside a flow and contextual dependencies between flows are seldom modelled together.

To address this gap, the thesis organises detection into two stages. Raw PCAP traffic is

first converted into aligned packet records and flow statistics by a custom Suricata output module. Stage 1 derives a flow representation from packet order, elapsed time, and flow statistics. A bounded start-time-ordered history of related representations is then used by Stage 2 to refine the flow-level decision. Separating these operations permits intra-flow representation learning and inter-flow reasoning to be assessed independently without changing the final prediction granularity.

The central research problem of this thesis is therefore:

How can network intrusion detection benefit from packet-level temporal modelling and cross-flow contextual modelling while preserving the hierarchical structure of network traffic?

To answer this question, the thesis studies two research questions:

- **RQ1:** Does time-aware packet-sequence modelling produce more informative flow representations than flow statistics alone?
- **RQ2:** Does incorporating cross-flow context improve flow-level intrusion detection compared with independent per-flow classification?

The thesis makes four principal contributions:

- An aligned PCAP-to-learning pipeline built around a custom Suricata output module, with explicit separation among predictive variables, context identifiers, and alert-derived targets.
- A Stage 1 representation that integrates packet position, cumulative elapsed time, mask-aware self-attention, and bounded flow-conditioned token modulation.
- A Stage 2 target-query attention model that draws evidence from relation-specific, start-time-ordered histories while excluding later flows and historical labels.
- External evaluation on chronological windows from the public CICIDS2017 Wednesday capture and on a separate company-provided server capture, complementing the source-domain chronological test.

Given the low prevalence of the positive class, macro-F1, PR-AUC, attack-class performance, and false-positive rate carry more interpretive weight than accuracy alone.

2 Related Work

Research on network intrusion detection spans manually specified signatures and data-driven models learned from packet traces, flow records, or ordered network events. The literature can be organised along two dimensions: representation granularity and contextual scope. Inputs range from packet bytes and header sequences to flow statistics and learned embeddings. Most systems assign labels independently to individual packets or flows; comparatively fewer studies represent temporal or relational dependencies across several flows.

2.1 Signature-Based Detection and Flow-Statistics Models

Expert-authored rules are the basis of signature-driven NIDSs. Snort introduced a lightweight, extensible design for real-time packet inspection [1]; Suricata extends this operational model with multi-threaded processing and programmable logging [5]. Their alerts are interpretable and rule behaviour can be controlled, which preserves their operational relevance. Coverage, however, depends on maintaining an appropriate signature set, and new or behaviourally altered attacks may fall outside it. Sommer and Paxson place this limitation in a broader methodological context: learning-based NIDS research must account for both the open-world nature of traffic and the semantic distance between laboratory benchmarks and deployed networks [2].

Classical machine-learning pipelines usually group packets into bidirectional flows and describe each flow through duration, volume, rate, TCP-flag, and inter-arrival summaries. Ahmed et al. and Buczak and Guven review the use of decision trees, support-vector machines, nearest-neighbour methods, clustering, and ensembles for anomaly and intrusion detection [8, 7]; conventional models can also benefit from feature selection and hybrid classification [16]. Aggregation converts a variable-length conversation into an efficient fixed-dimensional vector. That convenience entails a loss of process information: similar totals may arise from different packet orders, directional changes, burst structures, or temporal gaps. A model restricted to flow statistics cannot observe those generating dynamics directly.

2.2 Deep Learning for Packet and Flow Sequences

Deep architectures learn non-linear traffic representations and thereby reduce reliance on manually selected feature combinations. Recurrent NIDSs maintain a hidden state over ordered observations; Yin et al., for example, demonstrated that recurrent neural networks can capture sequential dependencies in intrusion data [9]. Shone et al. paired a non-symmetric deep autoencoder with a supervised classifier to derive compact representations [10]. Kitsune pursued lightweight online anomaly detection through an ensemble of incrementally processed autoencoders [11]. Local pattern extraction and recurrent temporal modelling have also been combined in CNN-LSTM hybrids [17].

Architecture alone does not determine whether temporal structure is learned; the construction of the sequence is equally important. An LSTM or GRU is sometimes applied across the dimensions of one flow-statistics vector, so the resulting “sequence” represents feature order rather than chronological packets or flows. Feature interactions may be

learned in this setting, but network time is not necessarily represented. NetSentry adopts a more direct temporal formulation by processing ordered observations for early detection of large-scale attacks [18]. Its findings support the value of temporal correlation beyond independent record classification. Recurrent computation nevertheless proceeds sequentially, and compressing a long or irregularly sampled history into one final state can create an information bottleneck.

2.3 Transformer-Based Traffic Representation

By replacing recurrent transitions with self-attention, Transformers permit direct interaction between distant tokens and support parallel training [12]. Both properties are relevant when evidence is separated by numerous packets or flows. RTIDS applied this mechanism to robust intrusion classification on flow-oriented benchmarks [13]. Long et al. subsequently evaluated an encoder-based Transformer for cloud-network intrusion detection [19], whereas IDS-INT combined Transformer transfer learning, imbalance handling, and downstream neural components [17].

GTID combines packet- and session-level features in a time-aware Transformer whose temporal coordinate is derived from inter-packet intervals rather than ordinal position alone [14]. FlowTransformer takes a complementary, modular approach, enabling input encodings, Transformer blocks, classification heads, parameter counts, and runtime to be examined on flow-based NIDS datasets [15]. The Memory Flow Transformer extends attention to longer flow histories and complex relations in large sequential inputs [20]. Together, these studies show that dependencies lost in a single flow-statistics vector can be recovered by attention-based models. They also make clear that encoding, pooling, sequence definition, and computational cost are at least as consequential as the choice of model family.

Three unresolved distinctions motivate the present hierarchy. Ordinal adjacency does not reveal whether two packets are separated by microseconds or seconds. Packet sequences and flow statistics also describe different properties of the same connection. Finally, even a rich representation of one flow cannot establish on its own whether that flow belongs to a distributed host-level pattern. The proposed design treats these as separate representational requirements.

2.4 Self-Supervised and Encrypted-Traffic Representation Learning

As encryption restricts payload inspection, traffic analysis has increasingly turned to packet sizes, directions, timing, header metadata, and learned latent representations. Surveys of encrypted-traffic analysis document this change and the expanding role of deep pre-training [3, 4]. ET-BERT transfers bidirectional Transformer pre-training to contextualised datagram representations for encrypted-traffic classification [21]. Masked autoencoders learn from unlabelled records by reconstructing concealed parts of an input [22, 23], while contrastive objectives improve label efficiency by aligning compatible traffic views and separating incompatible ones [24].

Recent work has also widened the range of admissible representations. CoTNeT applies contextual Transformer processing to encrypted-traffic features [25]; DoLLM reformulates flow records for large-language-model-based DDoS detection [26]. Both illustrate that plaintext payloads are not a prerequisite for useful representation learning. Nevertheless,

encrypted-traffic classification should not be conflated with intrusion detection. An application’s identity can remain stable even when malicious behaviour is sparse, evolving, or distributed over several connections.

2.5 Temporal, Relational, and Cross-Flow Context

Conditional independence between flows is a convenient assumption for batching and online inference, but many attack processes violate it. Scanning, password guessing, command and control, DDoS, and lateral movement may each generate a collection of individually ambiguous flows whose shared source, destination, endpoint, or temporal pattern is informative. NetSentry demonstrates the value of preserving observation order [18]. Graph-based NIDSs instead encode host and flow relations explicitly: Deng et al. use graph convolution over flow topology for label-limited IoT detection [27], and Xu et al. combine graph neural networks with self-supervised learning for network-flow intrusion detection [28]. Zhong et al. survey this expanding area and emphasise the capacity of graph models to represent structural dependence [29].

Relational inductive biases are expressive, although maintaining a large communication graph can increase memory demand, latency, and deployment complexity. Window-based sequences occupy a different point in this design space. One recent Transformer groups IoT flows into temporal windows and represents every flow as a token, allowing inter-flow dependence to be learned through self-attention [30]. A review of contextualised NetFlow NIDSs further distinguishes temporal, relational, multimodal, and multi-resolution context. It also stresses that chronological ordering and split isolation must be preserved during evaluation[31].

The modelling scope of representative Transformer-based approaches is contrasted in Table 2.1. Numerical ranking would be inappropriate because the underlying datasets, labels, preprocessing procedures, and evaluation protocols differ. In this comparison, *explicit elapsed time* requires direct encoding of timestamps or inter-arrival intervals; ordinal position or temporal-window membership alone is not sufficient.

TABLE 2.1: Qualitative scope of representative Transformer-based traffic models.

(a) Representation and temporal modelling			
Method	Primary input	Packet order	Elapsed time
FlowTransformer [15]	Flow records and statistics	No	Not explicitly encoded
GTID [14]	Packet headers and payload n-grams within a session	Yes	Inter-packet intervals
ET-BERT [21]	Datagram bytes grouped into BURSTs	Byte and BURST order	Order only; no packet intervals
Flow-to-window Transformer [30]	Flow records in fixed-duration windows	No	Window order; no packet intervals
Proposed	Packet-header sequence and flow statistics	Yes	Packet intervals

(b) Context, payload use, and task			
Method	Inter-flow context	Payload use	Primary task
FlowTransformer [15]	Ordered flow sequence	None	Flow-level NIDS
GTID [14]	None	Payload n-grams	NIDS
ET-BERT [21]	None	Encrypted bytes	Encrypted-traffic classification
Flow-to-window Transformer [30]	Temporal window	None	IoT NIDS and traffic classification
Proposed	Time, source, destination, or endpoint	No payload content	Flow-level NIDS

Notes: Entries describe the principal configuration reported in each study; extensible frameworks may support other variants. “No payload content” excludes direct payload bytes but not scalar metadata such as payload length.

This qualitative contrast identifies a gap without treating results from incompatible experimental settings as directly rankable. FlowTransformer and the flow-to-window model capture dependencies among flow records but do not reconstruct the packet dynamics within a flow. GTID represents both packet order and inter-packet time, although it processes one session at a time and incorporates payload-derived n-grams. ET-BERT learns byte- and BURST-level representations from encrypted raw traffic, but is primarily designed for encrypted-traffic classification rather than cross-flow intrusion detection. Within the selected comparison, only the proposed framework combines payload-content-independent packet modelling, explicit elapsed time, flow-statistics conditioning, and configurable context across flows. These complementary requirements motivate the separation of Stage 1 from Stage 2.

2.6 Research Gap

Flow statistics, sequential encoders, time-aware attention, self-supervised traffic representations, and graph- or window-based context have each proved useful. They are commonly investigated in isolation, however. Flow-statistics models trade packet dynamics for ef-

iciency; packet-sequence encoders generally stop at independent flow classification; and cross-flow models often begin with raw flow-statistics vectors rather than representations learned from timing-aware packet sequences.

The proposed hierarchy assigns these two scales of evidence to different stages. Packet order and elapsed time are represented within Stage 1 and conditioned by flow statistics. Stage 2 operates on the resulting embeddings under explicit temporal and endpoint-based context policies. In this way, *intra-flow* packet dynamics are connected to *inter-flow* attack context without collapsing them into one input representation. Separate ablations then test the contribution of time-aware packet modelling and the additional value of contextual flow history.

3 Dataset Construction and Preprocessing

Two synchronised views of the traffic are required: an ordered set of packet records for every flow and a single flow-statistics record describing that flow. The raw CSV schema is kept distinct from the Stage 1 predictor set and from identifiers used only to construct Stage 2 context, since linkage metadata need not be suitable predictive variables. Alignment and sequence assembly are deterministic, whereas every transformation estimated from data is fitted on the training partition. This organisation supports reproducibility and limits information leakage during NIDS evaluation [32, 33, 34, 35].

3.1 Data Provenance and Scope

Experiments are conducted on `ar002_et12_20260511_001.pcap`, an approximately 8.62 GB packet capture supplied by the company. Organisational staff acquired the trace from the company network with an internal traffic-collection tool; no laboratory traffic was generated for this thesis. CICIDS-style datasets schedule benign profiles and attacks under controlled conditions [36]. By contrast, the present trace records enterprise traffic as it was visible at the collection point. This improves environmental realism but also binds the observed traffic mix, host population, sensor visibility, and attack coverage to one organisation and capture period.

Because the dataset is proprietary, it is not presented as a public benchmark. Sensitive network identifiers, notably IP addresses and ports, remain in the source PCAP and CSV files; only aggregate quantities are reported in the thesis. Before any artefact leaves the authorised environment, host identifiers should be replaced consistently, superfluous endpoint fields removed, and the result reviewed under company data-governance policy. Stage 2 relies on equality relations between hosts, so stable pseudonyms are preferable to removing every identifier, although the pseudonym map itself need not be disclosed. Raw addresses are still retained by the current extraction code in local metadata. Pseudonymisation is therefore a requirement for future release, not a property claimed for the current CSV files.

3.2 External Evaluation Datasets

Two datasets that are disjoint from the source capture are used to examine transfer beyond its internal chronological test. The first is the publicly available CICIDS2017 Wednesday working-hours capture, corresponding to Wednesday, 5 July 2017 [36]. CICIDS2017 is not company-provided traffic and is treated as a public cross-dataset test.

Compatible external-test artefacts for Stage 1 are ordered chronologically and divided into non-overlapping windows of 20,000 flows. To preserve the qualitative class-imbalance condition of the source task, external windows are retained when label 1 remains the minority class and label 0 the majority class. Under this prevalence-based criterion, the evaluated windows are `w01`, `w02`, `w06`, `w07`, and `w08`, contributing 100,000 flows in total. The criterion concerns class composition rather than model performance. Window-level reporting is retained because aggregate performance alone can conceal temporal variation in false-positive and false-negative behaviour.

The second external dataset, `firewall_test_20250731_160700`, was supplied by the company but collected from a server environment different from that of `ar002_et12_20260511_001`. It contains 38,513 evaluated flows and is used as a cross-server organisational test. It is neither part of CICIDS2017 nor described as a public benchmark. This distinction separates public cross-dataset transfer from transfer to a different server within the organisational setting.

The external datasets are evaluation resources rather than extensions of the source training partition. Their class prevalence differs from that of the source capture: the five prevalence-compatible CICIDS2017 windows contain between 13.34% and 18.20% positive flows, whereas the separate-server capture contains 3.285%. Metrics are consequently interpreted within each dataset rather than through accuracy alone. Selecting CICIDS2017 windows in which label 1 remains the minority improves comparability with the imbalanced source task, but it also limits the external claim to windows satisfying that condition rather than the complete Wednesday capture. Differences in capture environment, traffic composition, labelling provenance, and class balance mean that an external score is evidence of transfer under the evaluated pipeline, not proof of invariant performance across arbitrary networks.

3.3 PCAP-to-CSV Extraction

Suricata, an open-source network threat detection engine, is used to replay the PCAP [5]. The custom C module at `suricata-plugin/custom-out.c` registers separate packet and flow loggers through Suricata’s low-level output interfaces. Both CSV views are emitted during the same pass. Consequently, packet-to-flow membership does not need to be reconstructed by another tool, and the packet and flow tables retain a common bidirectional `flow_id`.

The construction path is shown in Figure 3.1. Packet rows retain microsecond timestamps and their order within each flow. In parallel, online aggregates are maintained by the flow logger and written once a flow completes. The broad feature families follow CICFlowMeter/CICIDS-style data. Additional indicators record whether initial TCP-window measurements are available in the forward and backward directions [36].

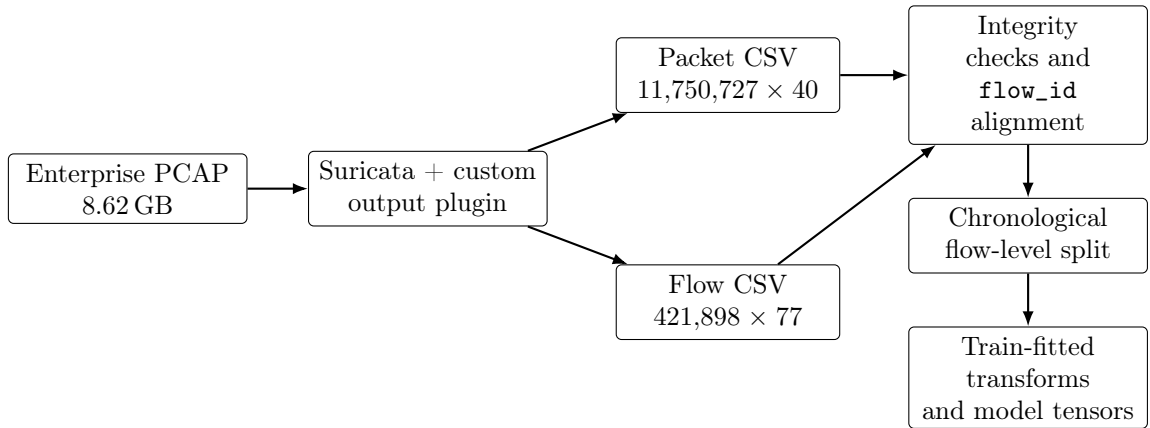


FIGURE 3.1: Construction and preprocessing path from the proprietary PCAP to Stage 1 tensors and Stage 2 metadata.

Neither learning stage receives raw application payload bytes. Although `payload_len` is present, it records only a scalar length derived from packet metadata and contains no

application content. The detector is therefore independent of payload content, but not of every payload-related measurement. Suricata signatures, rule identifiers, alert text, and alert counts are likewise excluded from the predictor set.

3.4 Alert-Derived Label Construction

Binary targets are derived by the custom logger from Suricata’s alert state. If $a(p_i)$ is the number of alerts associated with packet p_i , its packet-level target is

$$y_i^{\text{pkt}} = \mathbb{I}[a(p_i) > 0], \quad (3.1)$$

where $\mathbb{I}[\cdot]$ denotes the indicator function. A positive flow contains at least one packet that triggered an alert:

$$y_f = \max_{p_i \in \mathcal{P}_f} y_i^{\text{pkt}}, \quad (3.2)$$

where $\mathcal{P}_f$ contains all packets assigned to flow f . Accordingly, `packet_label=1` identifies an alert-triggering packet, whereas `label=1` marks an alert-associated flow.

These programmatically assigned targets are *weak labels*, not independently verified ground truth [37]. Label 0 signifies only that the configured Suricata rules emitted no alert; benignity is not thereby established. Positive targets similarly inherit false alarms from the deployed ruleset. The reported experiments therefore evaluate the learning and contextualisation of behaviour associated with Suricata alerts. They do not establish guaranteed coverage of every intrusion or zero-day attack. Errors in the labelling process may influence both model fitting and test estimates and must be considered when the findings are interpreted [38].

Direct target leakage is prevented by excluding `packet_label`, `label`, and all Suricata alert metadata from model inputs. Exact replication of the weak-labelling function would additionally depend on the Suricata version, a ruleset snapshot, the custom-module revision, its configuration, and the PCAP checksum. Where this provenance was not retained, the omission is reported in Chapter 5 rather than filled retrospectively.

3.5 Raw Dataset Composition

Table 3.1 reports the dimensions of the aligned outputs. There are 11,750,727 packet rows across 40 columns and 421,898 flow rows across 77 columns. Before truncation, this corresponds to a mean of approximately 27.85 logged packets per flow. The mean should not be read as evidence of uniform flow length: packet counts are typically heavy-tailed, making an explicit upper bound on sequence length necessary.

TABLE 3.1: Raw extracted dataset summary.

Data view	Rows	Columns	Primary role
Packet CSV	11,750,727	40	Ordered packet-header, protocol, direction, and inter-arrival observations within flows
Flow CSV	421,898	77	One flow-level target, flow statistics, and context-construction metadata per flow

Among the 421,898 flow records, 405,254 carry label 0 and 16,644 carry label 1, as detailed in Table 3.2. Hence, the positive prevalence and class-imbalance ratio are

$$\pi_1 = \frac{16,644}{421,898} = 0.03945 \quad (3.945\%), \quad (3.3)$$

$$\text{IR} = \frac{405,254}{16,644} = 24.35 : 1. \quad (3.4)$$

TABLE 3.2: Flow-level alert-derived label distribution.

Label	Interpretation	Count	Proportion
0	No Suricata alert observed in the flow	405,254	96.055%
1	At least one alert-triggering packet	16,644	3.945%
Total		421,898	100.000%

At this prevalence, predicting label 0 for every flow would yield approximately 96.055% accuracy and zero positive recall. Accuracy and frequency-weighted averages are therefore treated as secondary. Macro-precision, macro-recall, macro-F1, ROC-AUC, PR-AUC, confusion-matrix counts, and false-positive rate provide the main evidence. PR-AUC is especially informative for a rare positive class [39]. Operationally, false positives also warrant direct attention because even a small fraction of 405,254 negative flows can translate into a large number of alerts.

3.6 Raw Packet and Flow Schemas

All raw columns are grouped by function in Tables 3.3 and 3.4. These groups describe the extracted schema and should not be interpreted as the final predictor set.

TABLE 3.3: Complete packet CSV schema grouped by function.

Group	Raw columns
Record identity and time	record_type, packet_id, timestamp_us, ts_sec, ts_usec, flow_id
Endpoints and protocol	direction, ip_version, src_ip, dst_ip, src_port, dst_port, protocol, l7_proto
Length and header metadata	pkt_len, ip_len, payload_len, ttl_or_hop_limit, tcp_window, tcp_header_len, udp_len
Transport indicators	has_tcp, has_udp, has_icmp
TCP state and flags	tcp_seq, tcp_ack, tcp_flags, tcp_fin, tcp_syn, tcp_rst, tcp_psh, tcp_ack_flag, tcp_urg, tcp_ece, tcp_cwr
ICMP metadata	icmp_type, icmp_code
Inter-arrival time	flow_iat_us, direction_iat_us
Target	packet_label

The two packet-level timing fields are not interchangeable. `flow_iat_us` measures elapsed time since the preceding packet in the bidirectional flow; `direction_iat_us` refers

instead to the preceding packet travelling in the same direction. Values are expressed in microseconds and set to zero when no applicable predecessor exists.

TABLE 3.4: Complete flow CSV schema grouped by function.

Group	Raw columns
Identity and time	flow_id, flow_start_timestamp_us, flow_end_timestamp_us
Endpoints and protocol	source_ip, source_port, destination_ip, destination_port, protocol
Directional volume	total_fwd_packets, total_backward_packets, total_length_of_fwd_packets, total_length_of_bwd_packets
Directional length statistics	fwd_packet_length_max, fwd_packet_length_min, fwd_packet_length_mean, fwd_packet_length_std, bwd_packet_length_max, bwd_packet_length_min, bwd_packet_length_mean, bwd_packet_length_std
Overall rate and timing	flow_duration, flow_bytes_per_s, flow_packets_per_s, flow_iat_mean, flow_iat_std, flow_iat_max, flow_iat_min
Directional inter-arrival time	fwd_iat_total, fwd_iat_mean, fwd_iat_std, fwd_iat_max, fwd_iat_min, bwd_iat_total, bwd_iat_mean, bwd_iat_std, bwd_iat_max, bwd_iat_min
Directional flags, headers, and rates	fwd_psh_flags, bwd_psh_flags, fwd_urg_flags, bwd_urg_flags, fwd_header_length, bwd_header_length, fwd_packets_per_s, bwd_packets_per_s
Overall packet-length distribution	min_packet_length, max_packet_length, packet_length_mean, packet_length_std, packet_length_variance
TCP flag counts	fin_flag_count, syn_flag_count, rst_flag_count, psh_flag_count, ack_flag_count, urg_flag_count, cwe_flag_count, ece_flag_count
Ratios, segments, and initial windows	down_up_ratio, average_packet_size, avg_fwd_segment_size, avg_bwd_segment_size, has_init_win_bytes_forward, init_win_bytes_forward, has_init_win_bytes_backward, init_win_bytes_backward, act_data_pkt_fwd, min_seg_size_forward
Active and idle periods	active_mean, active_std, active_max, active_min, idle_mean, idle_std, idle_max, idle_min
Target	label

TCP-specific fields are zero-filled for non-TCP packets, with `has_tcp` distinguishing an inapplicable field from a genuine observed zero. UDP and ICMP presence indicators serve the same purpose. At flow level, `has_init_win_bytes_forward` and `has_init_win_bytes_backward` mark the availability of their corresponding initial-window measurements; a missing measurement is stored as zero.

3.7 Integrity Checks and Packet–Flow Alignment

Preprocessing begins by normalising column names and discarding `record_type`, a logging descriptor with no predictive role. Infinite values are converted to missing entries. Rows are

rejected when `flow_id` is zero or missing; valid flow rows also require `flow_start_timestamp_us`, while packet rows require `timestamp_us`, in each case with a nonzero, nonmissing value.

Only one effective flow row is retained for each `flow_id`. When duplicates occur, a stable sort by identifier and start time is applied, the first record is kept, and the number of discarded duplicates is logged. The identifier sets of the packet and flow tables are then intersected so that unmatched rows can be removed from both views. A final sort places packets in `flow_id` and `timestamp_us` order. Together, these operations establish the one-to-many relation

$$f \longleftrightarrow \{p_{f,1}, p_{f,2}, \dots, p_{f,n_f}\}, \quad t_{f,1} \leq t_{f,2} \leq \dots \leq t_{f,n_f}, \quad (3.5)$$

before splitting or feature transformation. Table 3.1 describes the CSV files as extracted, whereas post-cleaning counts for a particular run are determined from its saved split report.

3.8 Feature Selection and Leakage Control

Raw columns are not passed wholesale to the network; the input schema is controlled by configuration. Before one-hot expansion, the principal Stage 1 setup uses the 23 packet predictors listed in Table 3.5. Numerical variables encode packet and header dimensions, categorical variables describe protocol state, and binary indicators retain direction, transport presence, and individual TCP flags.

TABLE 3.5: Packet predictors used by the principal Stage 1 configuration.

Type	Selected raw columns
Numerical (7)	<code>pkt_len</code> , <code>ip_len</code> , <code>payload_len</code> , <code>ttl_or_hop_limit</code> , <code>tcp_window</code> , <code>tcp_header_len</code> , <code>udp_len</code>
Categorical (4)	<code>ip_version</code> , <code>protocol</code> , <code>icmp_type</code> , <code>icmp_code</code>
Binary (12)	<code>direction</code> , <code>has_tcp</code> , <code>has_udp</code> , <code>has_icmp</code> , <code>tcp_fin</code> , <code>tcp_syn</code> , <code>tcp_rst</code> , <code>tcp_psh</code> , <code>tcp_ack_flag</code> , <code>tcp_urg</code> , <code>tcp_ece</code> , <code>tcp_cwr</code>

Stage 1 excludes raw IP addresses and ports, packet and flow identifiers, absolute timestamps, TCP sequence and acknowledgement numbers, the combined TCP-flag integer, `17_proto`, and both targets. Their removal discourages memorisation of site-specific endpoints and prevents identifiers or capture position from becoming target shortcuts. Relative timing is retained separately through a tensor derived from `flow_iat_us`; the absolute packet timestamp is not supplied as an ordinary feature.

When flow-statistics fusion is enabled, a configured subset of Table 3.4 is processed through a separate branch. The principal subset covers duration, directional packet and byte totals, length distributions, rates, inter-arrival summaries, flag counts, header lengths, segment and window measurements, and selected active-period statistics. Availability of the two initial-window measurements is represented by binary indicators. Endpoint identities and ports, absolute flow times, and `label` are omitted. Since the packet-only ablation disables this branch, presence in the raw schema does not imply use in every experiment.

Three metadata fields accompany every Stage 1 embedding: `flow_start_timestamp_us`, `source_ip`, and `destination_ip`. They define relations for Stage 2 and are not Stage 1 predictors. Host equality determines source-host, destination-host, and endpoint contexts, while start time establishes ordering. Addresses are never one-hot encoded as semantic features, and historical labels are not retrieved into context. The required relations are thereby preserved without directly training on host identity.

3.9 Train-Fitted Feature Transformations

Every data-dependent transformation is estimated from the training partition and subsequently applied unchanged to validation and test records. Estimating clipping limits, scaling statistics, or category vocabularies from the complete dataset would expose information from later partitions [40]. Packet and flow variables follow pipelines of the same form, but their fitted parameters remain separate.

Numerical features. After conversion to numeric form, non-finite entries are marked as missing. Selected nonnegative variables with heavy-tailed distributions undergo the transform

$$\tilde{x} = \log(1 + \max(x, 0)). \quad (3.6)$$

Configured variables include packet and payload lengths, TCP window, header length, UDP length, and numerous flow-level counts, durations, byte totals, rates, and inter-arrival summaries. Zero imputation is applied after the logarithmic transform.

For numerical variable j , lower and upper clipping limits are the training quantiles $q_{0.001,j}^{\text{tr}}$ and $q_{0.999,j}^{\text{tr}}$. Winsorisation in every partition is defined by

$$x_{ij}^{\text{clip}} = \min(\max(\tilde{x}_{ij}, q_{0.001,j}^{\text{tr}}), q_{0.999,j}^{\text{tr}}). \quad (3.7)$$

Robust scaling is then applied in the principal configuration:

$$x_{ij}^{\text{scaled}} = \frac{x_{ij}^{\text{clip}} - \text{median}_{\text{tr}}(x_j)}{\text{IQR}_{\text{tr}}(x_j)}, \quad (3.8)$$

where the denominator is computed from the training interquartile range. Logarithmic compression, quantile clipping, and robust scaling limit the influence of extreme bursts without drawing statistics from the test partition.

Categorical features. An explicit MISSING token represents absent categorical values. The one-hot vocabulary is learned from training categories; categories encountered only during validation or testing are ignored, leaving the input dimension unchanged.

Binary features. Binary fields are first converted to numbers. Missing and non-finite entries become zero, as do all non-positive values, while positive entries become one. No one-hot encoding, logarithmic transform, clipping, or scaling is applied to these indicators.

3.10 Packet-Sequence Construction

Packets in aligned flow f are sorted by `timestamp_us`. Under the principal `head` policy, at most $L = 128$ packets are retained. Flows longer than L contribute their first 128 observations; shorter sequences are padded with zeros on the right. Valid positions are recorded in a Boolean mask so that padding is excluded from attention and pooling. The resulting tensor shapes are

$$\mathbf{X} \in \mathbb{R}^{N \times L \times d_p}, \quad \mathbf{T} \in \mathbb{R}^{N \times L}, \quad \mathbf{M} \in \{0, 1\}^{N \times L}, \quad (3.9)$$

with d_p denoting the transformed packet-feature dimension after one-hot encoding.

Nonnegative within-flow inter-arrival times define the timing tensor:

$$T_{f,i} = \log(1 + \max(\Delta t_{f,i}, 0)), \quad \Delta t_{f,i} = \text{flow_iat_us}(p_{f,i}). \quad (3.10)$$

This tensor enters the time-aware encoder separately; absolute timestamps are not concatenated to the packet features.

Two additional policies support the truncation ablation. **head_tail** combines the first $\lfloor L/2 \rfloor$ packets with the final $L - \lfloor L/2 \rfloor$ packets. **random** samples L positions without replacement from a deterministic seed derived from **flow_id** and the global seed, after which the selected packets are returned to chronological order. Separate experiments consider $L \in \{8, 16, 32, 64, 128, 256\}$. Only the retained evidence and associated computation change; flow membership in each split remains fixed.

3.11 Chronological Train–Validation–Test Partition

Unique flows are stably ordered by **flow_start_timestamp_us** before a flow-level split is made. The earliest 70% form the training partition, the following 10% form validation, and the final 20% are reserved for testing. With **stratify=false**, these chronological boundaries are not adjusted to equalise class proportions. For the 421,898 aligned records, this procedure yields the partition sizes in Table 3.6.

TABLE 3.6: Principal chronological flow partition.

Partition	Ratio	Number of flows	Temporal role
Training	70%	295,329	Earliest flows
Validation	10%	42,189	Intermediate flows
Test	20%	84,380	Latest flows
Total	100%	421,898	

Every packet follows the partition assignment of its parent **flow_id**; no flow can span two partitions. Only training-flow packets and statistics are used to fit preprocessing. Validation data support hyperparameter choice, early stopping, and decision-threshold selection, while final evaluation is confined to the test partition. Relative to random row splitting, chronological separation more closely reflects deployment on later traffic and reduces leakage from distributing related packets or temporally adjacent behaviour across partitions [2, 40].

3.12 Stage 1 Artifacts Supplied to Stage 2

Once Stage 1 has been trained, exactly one embedding and one metadata record are exported for each flow in every partition. Metadata comprise **flow_id**, the target, partition name, **flow_start_timestamp_us**, and source and destination identifiers. The embedding provides Stage 2 content; time and host identity are used only to determine which earlier flows are eligible. Retaining the partition name guards against unintended context construction across split boundaries. Targets remain available for training and evaluation but never become context features.

Each Stage 1 run also records the selected features, encoder categories, clipping limits, scaling parameters, sequence policy, random seed, ordered flow identifiers, partition-level

class counts, model configuration, and checkpoint identity. These records link every Stage 2 result to the Stage 1 checkpoint and split manifest from which its embeddings originated.

3.13 Validity, Privacy, and Reproducibility Considerations

Although the resulting dataset supports evaluation on a large, temporally ordered enterprise trace, four constraints delimit the conclusions.

- L1: Alert-derived labels.** As detailed in Section 3.4, the targets are Suricata-derived weak labels; conclusions therefore concern alert-associated behaviour rather than independently verified attacks and benign traffic.
- L2: Limited multi-source scope.** Model-development data and ablation evidence come from one company source capture. The two external sources described in Section 3.2 broaden the evaluation scope but do not establish generalisation to arbitrary organisations, sensors, rulesets, or periods.
- L3: Capture visibility.** Only traffic observable at the collection point appears in the PCAP. Packet loss, asymmetric routing, encryption, and incomplete flows may change the extracted measurements. Where organisational policy allows, documentation should include the collection tool, interface location, capture interval, packet-loss report, and snap length.
- L4: Sensitive identifiers.** Stage 2 relations are currently formed from addresses retained in local metadata. Those addresses must be excluded from the thesis, shared CSVs, screenshots, and public metadata. Any released artefact requires stable pseudonymisation verified to preserve equality-based context membership.

Within these limits, the study evaluates a traceable learning pipeline on a large proprietary capture. Tensors, masks, flow statistics, start-time metadata, and relation identifiers together define the interface passed between the two model stages.

4 Proposed Methodology

Two complementary scales of traffic structure are represented by the detector. Stage 1 operates within a bidirectional flow, combining its ordered packets with flow statistics. Stage 2 relates the target flow to eligible historical flows. Sequential training and a fixed-dimensional Stage 1 interface keep intra-flow representation learning distinct from contextual reasoning across flows.

Although the architecture is derived from the Transformer [12], self-attention alone does not define the method. The method combines three mechanisms: (i) a cumulative-time coordinate for irregular packet arrival, (ii) an independently encoded flow-statistics branch that modulates packet tokens within a bounded range, and (iii) target-query attention from the current flow to a start-time-ordered history.

The distinction between packet order and elapsed time follows evidence from time-aware NIDS research [14]. Separating within-flow evidence from cross-flow behaviour also addresses attacks whose signal is dispersed over several connections and may be missed when flows are classified independently [18, 15, 30].

The high-level data path appears in Figure 4.1. A time-aware intra-flow Transformer converts packet records into one embedding for each flow. A sequence of these embeddings is then contextualised in Stage 2 to produce a binary decision for the target flow. Figure 4.2 gives a more detailed view of flow-statistics conditioning and target-query attention.

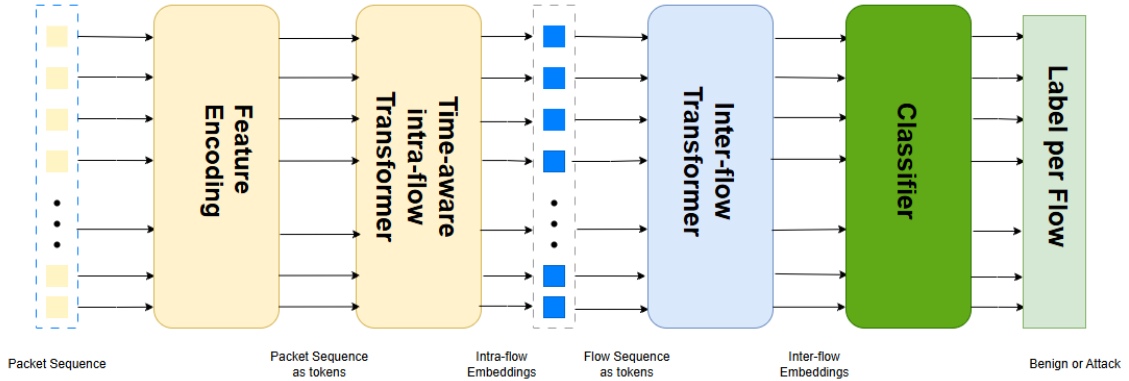


FIGURE 4.1: High-level architecture of the hierarchical detector. Stage 1 transforms a packet sequence into an intra-flow representation, and Stage 2 contextualises the resulting flow sequence before classifying the target flow.

4.1 Architectural Overview

Flow i is represented by a selected and padded packet-feature matrix $\mathbf{X}_i \in \mathbb{R}^{L \times d_p}$, a packet inter-arrival tensor $\mathbf{u}_i \in \mathbb{R}^L$, a validity mask $\mathbf{m}_i \in \{0, 1\}^L$, a flow-statistics vector $\mathbf{s}_i \in \mathbb{R}^{d_f}$, and a binary target $y_i \in \{0, 1\}$. The principal configuration uses $L = 128$. Stage 1 computes

$$\left(\ell_i^{(1)}, \mathbf{z}_i^{(1)}\right) = F_{\theta_1}(\mathbf{X}_i, \mathbf{u}_i, \mathbf{m}_i, \mathbf{s}_i), \quad (4.1)$$

where $\ell_i^{(1)} \in \mathbb{R}^2$ denotes the Stage 1 logits, and $\mathbf{z}_i^{(1)} \in \mathbb{R}^d$ is the fused representation supplied to Stage 2. The principal model sets $d = 128$.

Once all Stage 1 embeddings are available, flows are stably ordered by start time and flow identifier. Given relation r and target i , the bounded start-time-ordered input to Stage 2 is

$$\mathbf{Z}_i^{(r)} = [\mathbf{z}_{j_1}^{(1)}, \dots, \mathbf{z}_{j_{K_i}}^{(1)}, \mathbf{z}_i^{(1)}], \quad j_k < i, \quad K_i \leq W - 1, \quad (4.2)$$

The current flow occupies the final valid position, and the principal setting uses $W = 128$ tokens including that target. Stage 2 computes

$$\ell_i^{(2,r)} = G_{\theta_2}(\mathbf{Z}_i^{(r)}, \boldsymbol{\mu}_i), \quad (4.3)$$

where left padding is excluded by $\boldsymbol{\mu}_i$. Applying softmax to $\ell_i^{(2,r)}$ yields the final class-1 probability.

As indicated in Figure 4.2, labels and Suricata alert fields serve only as targets and are absent from both encoders.

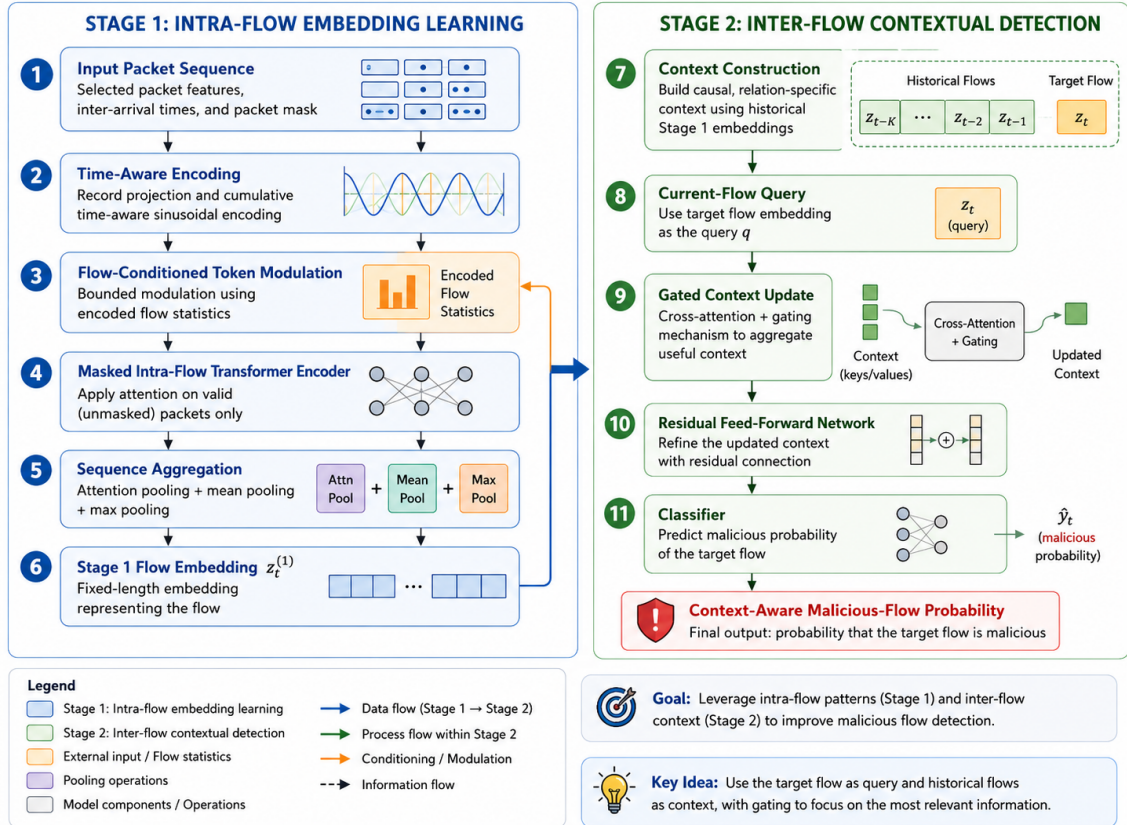


FIGURE 4.2: Detailed architecture of the proposed two-stage detector. The flow-statistics branch conditions Stage 1 packet tokens, while Stage 2 uses the target flow as the query and historical flows as keys and values.

4.2 Stage 1: Intra-Flow Time-Aware Representation

A variable-length packet sequence is reduced by Stage 1 to one flow embedding. The principal processing path comprises record projection, time-aware sinusoidal encoding, flow-conditioned token modulation, pre-normalised Transformer layers, hybrid mask-aware pooling, and a two-class head.

4.2.1 Packet-Record Projection

At packet position t , the preprocessed vector $\mathbf{x}_{i,t} \in \mathbb{R}^{d_p}$ is mapped into the model space:

$$\mathbf{e}_{i,t} = \text{LN}(W_p \mathbf{x}_{i,t} + \mathbf{b}_p), \quad \mathbf{e}_{i,t} \in \mathbb{R}^d. \quad (4.4)$$

An MLP projection is supported, although the principal configuration applies a single linear map followed by layer normalisation. As specified in Chapter 3, categorical fields have already been one-hot encoded, numerical variables transformed and robustly scaled, and binary indicators retained in $\{0, 1\}$.

4.2.2 Cumulative Time-Aware Encoding

Sinusoidal positional encoding captures ordinal packet order but assigns the same adjacency to gaps of microseconds and seconds. Stage 1 augments position with cumulative elapsed time. Preprocessing stores

$$u_{i,t} = \log(1 + \Delta\tau_{i,t}), \quad (4.5)$$

where $\Delta\tau_{i,t}$ is the nonnegative microsecond gap from the preceding packet in the same bidirectional flow. The model reconstructs and masks this interval as

$$\widehat{\Delta\tau}_{i,t} = m_{i,t} \max(0, \exp(u_{i,t}) - 1). \quad (4.6)$$

Cumulative time and its smoothed logarithmic form are then

$$c_{i,t} = \sum_{k=1}^t \widehat{\Delta\tau}_{i,k}, \quad (4.7)$$

$$p_{i,t} = \log(1 + c_{i,t} + \alpha), \quad \alpha = 10^{-7}. \quad (4.8)$$

With zero-based position $q_t = t - 1$, order and time define the coordinate $r_{i,t} = q_t + p_{i,t}$. Channel pair j is encoded by

$$\text{TE}_{i,t,2j} = \sin\left(\frac{r_{i,t}}{10000^{2j/d}}\right), \quad (4.9)$$

$$\text{TE}_{i,t,2j+1} = \cos\left(\frac{r_{i,t}}{10000^{2j/d}}\right). \quad (4.10)$$

Padding positions are assigned zero encodings. A valid packet token becomes

$$\tilde{\mathbf{e}}_{i,t} = \text{Dropout}(\mathbf{e}_{i,t} + m_{i,t} \text{TE}_{i,t}). \quad (4.11)$$

This formulation adapts the time-aware Transformer principle in GTID [14] to structured packet metadata without payload content. Two ablations isolate its coordinates: time-only encoding sets $r_{i,t} = p_{i,t}$, whereas position-only encoding sets $r_{i,t} = q_t$.

4.2.3 Flow-Statistics Encoder

A bottleneck encoder maps the separately preprocessed flow-statistics vector $\mathbf{s}_i$ to the packet-model dimension:

$$\mathbf{f}_i = \text{LN}_2(W_{f,2} \text{ Dropout}(\text{GELU}(\text{LN}_1(W_{f,1}\mathbf{s}_i + \mathbf{b}_{f,1}))) + \mathbf{b}_{f,2}). \quad (4.12)$$

Its principal hidden and output dimensions are 32 and 128, respectively. Restricting the flow-statistics branch in this way reduces the chance that a high-dimensional flow-statistics vector dominates evidence learned from packets.

4.2.4 Bounded Flow-Conditioned Token Modulation

Feature-wise linear modulation (FiLM) motivates the principal Stage 1 fusion mechanism: conditioning information generates affine parameters for individual features [41]. A small network derives the modulation parameters from the flow representation

$$[\boldsymbol{\gamma}_i; \boldsymbol{\beta}_i] = G_f(\mathbf{f}_i), \quad \boldsymbol{\gamma}_i, \boldsymbol{\beta}_i \in \mathbb{R}^d. \quad (4.13)$$

The unbounded flow-conditioned residual for packet token t is

$$\mathbf{r}_{i,t}^{\text{raw}} = \eta(\boldsymbol{\gamma}_i \odot \tilde{\mathbf{e}}_{i,t} + \boldsymbol{\beta}_i), \quad \eta = \sigma(a), \quad a_0 = -5, \quad (4.14)$$

where $\odot$ is element-wise multiplication and the initial scale is $\sigma(-5) \approx 0.0067$. A second constraint bounds this residual relative to the packet-token norm. For the configured maximum ratio $\rho = 0.13$,

$$\lambda_{i,t} = \min\left(1, \frac{\rho \|\tilde{\mathbf{e}}_{i,t}\|_2}{\|\mathbf{r}_{i,t}^{\text{raw}}\|_2 + \epsilon}\right), \quad (4.15)$$

$$\mathbf{e}_{i,t}^F = \tilde{\mathbf{e}}_{i,t} + \lambda_{i,t} \mathbf{r}_{i,t}^{\text{raw}}. \quad (4.16)$$

Zero initialisation is used for the layer that produces $\boldsymbol{\gamma}_i$ and $\boldsymbol{\beta}_i$. Training therefore starts from the packet-only pathway, after which a bounded flow-statistics correction can be learned. Such a constraint is useful when flow statistics are informative yet potentially noisy or capture-specific.

Because the flow-statistics vector is encoded separately and then modulates packet tokens before self-attention, the principal configuration constitutes *separate-branch conditional intermediate fusion*, rather than strict late fusion after pooling. Although a post-pooling gated alternative is implemented, the reported Scheme C results use the bounded token-FiLM operation in Equation 4.16.

4.2.5 Masked Transformer Encoder

Pre-normalised Transformer layers process the conditioned sequence $\mathbf{E}_i^F \in \mathbb{R}^{L \times d}$. At layer ℓ ,

$$\mathbf{A}_i^\ell = \text{MHA} \left(\text{LN}(\mathbf{H}_i^{\ell-1}), \text{LN}(\mathbf{H}_i^{\ell-1}), \text{LN}(\mathbf{H}_i^{\ell-1}); \mathbf{m}_i \right), \quad (4.17)$$

$$\overline{\mathbf{H}}_i^\ell = \mathbf{H}_i^{\ell-1} + \text{Dropout}(\mathbf{A}_i^\ell), \quad (4.18)$$

$$\mathbf{H}_i^\ell = \overline{\mathbf{H}}_i^\ell + \text{FFN} \left(\text{LN}(\overline{\mathbf{H}}_i^\ell) \right), \quad (4.19)$$

with $\mathbf{H}_i^0 = \mathbf{E}_i^F$. Scaled dot-product attention is applied in every head:

$$\text{Attn}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_h}} + M_i \right) V, \quad (4.20)$$

where M_i assigns negative infinity to padded keys. GELU is used in the feed-forward sublayer. The principal encoder contains two layers and four attention heads, with model dimension 128, feed-forward dimension 256, and dropout 0.15.

Stage 1 does not apply a causal mask, so every retained packet in the completed or truncated flow may attend to every other retained packet. The stage should therefore be interpreted as flow-level classification, not packet-by-packet early detection. Padding is excluded both as an attention key and during pooling.

4.2.6 Hybrid Mask-Aware Pooling

Three complementary summaries reduce the packet states. First, each valid token receives a learned attention score:

$$a_{i,t} = \mathbf{w}_{a,2}^\top \text{GELU} (W_{a,1} \mathbf{h}_{i,t} + \mathbf{b}_{a,1}) + b_{a,2}, \quad (4.21)$$

$$\omega_{i,t} = \frac{m_{i,t} \exp(a_{i,t})}{\sum_{k=1}^L m_{i,k} \exp(a_{i,k})}, \quad (4.22)$$

$$\mathbf{z}_i^{\text{att}} = \sum_{t=1}^L \omega_{i,t} \mathbf{h}_{i,t}. \quad (4.23)$$

Mask-aware mean and maximum summaries are

$$\mathbf{z}_i^{\text{mean}} = \frac{\sum_{t=1}^L m_{i,t} \mathbf{h}_{i,t}}{\max(1, \sum_{t=1}^L m_{i,t})}, \quad (4.24)$$

$$\mathbf{z}_i^{\text{max}} = \max_{t:m_{i,t}=1} \mathbf{h}_{i,t}. \quad (4.25)$$

All three summaries contribute to the Stage 1 embedding:

$$\mathbf{z}_i^{(1)} = \text{Dropout} \left(\text{GELU} \left(\text{LN} \left(W_o [\mathbf{z}_i^{\text{att}}; \mathbf{z}_i^{\text{mean}}; \mathbf{z}_i^{\text{max}}] + \mathbf{b}_o \right) \right) \right). \quad (4.26)$$

Attention pooling can concentrate on a few informative packets, the mean captures overall behaviour, and the maximum retains strong local activations. Projecting their concatenation back to dimension d gives Stage 2 a fixed-size vector for every flow.

Stage 1 classification logits are

$$\ell_i^{(1)} = h_1(\mathbf{z}_i^{(1)}), \quad \mathbf{p}_i^{(1)} = \text{softmax}(\ell_i^{(1)}). \quad (4.27)$$

Export occurs before the final two-class layer, preserving a multi-dimensional representation for contextual refinement instead of reducing the flow to one probability.

4.2.7 Stage 1 Fusion Variants

Table 4.1 defines three controlled variants. Unless stated otherwise, packet features, sequence policy, data split, Transformer capacity, optimisation procedure, and classifier are held constant across them.

TABLE 4.1: Stage 1 packet/flow integration variants.

Scheme	Input structure	Integration mechanism
A	Packet + repeated flow statistics	The transformed flow-statistics vector is concatenated to every packet before record projection.
B	Packet only	The flow-statistics branch is disabled; this isolates packet order and time-aware encoding.
C	Packet + separate flow branch	The flow-statistics vector is encoded once and conditionally modulates packet tokens through bounded token-FiLM. This is the principal proposed Stage 1 configuration.

4.3 Stage 2: Start-Time-Ordered Inter-Flow Modelling

Stage 2 tests whether recent related behaviour changes the interpretation of a target flow. Its inputs are restricted to Stage 1 embeddings and context metadata; raw packet fields, flow statistics, and historical labels are not loaded.

4.3.1 Embedding Interface and Chronological Order

For each split, Stage 1 exports

$$\left(\text{flow_id}_i, y_i, \mathbf{z}_i^{(1)}, t_i^0, u_i, v_i, \rho_i \right), \quad (4.28)$$

where t_i^0 gives the start time, u_i and v_i identify source and destination, and ρ_i records split membership. Stage 2 verifies identifier uniqueness, concatenates the split files, and applies a stable sort on $(t_i^0, \text{flow_id}_i)$. Embeddings and metadata are reordered together to preserve alignment.

A deterministic loader without sampling is used for embedding export. The weighted training sampler draws with replacement and would otherwise repeat some flows while omitting others; it is therefore restricted to optimisation batches.

4.3.2 Start-Time-Ordered Relation-Specific Contexts

Context construction proceeds once through the sorted flows in temporal order. Eligible earlier indices are retrieved before target i is added to any history. Table 4.2 defines the four relations considered.

TABLE 4.2: Stage 2 context relations.

Method	Eligibility condition for $j < i$	Interpretation
<code>time_only</code>	Always eligible	Most recent flows globally, regardless of end-point identity.
<code>source_host</code>	$u_j = u_i$	Earlier flows initiated by the same source identifier.
<code>destination_host</code>	$v_j = v_i$	Earlier flows directed to the same destination identifier.
<code>endpoint</code>	$\{u_j, v_j\} \cap \{u_i, v_i\} \neq \emptyset$	Earlier flows sharing at least one endpoint in either role.

For a given relation r , define

$$\mathcal{A}_i^{(r)} = \{j < i : R_r(i, j) = 1\}. \quad (4.29)$$

Split-policy filtering and deduplication are applied before retaining the latest $W - 1$ indices:

$$\mathcal{C}_i^{(r)} = \text{Recent}_{W-1} \left(\mathcal{A}_i^{(r)} \right). \quad (4.30)$$

Appending the current flow yields Equation 4.2. Histories are not updated until this sequence has been formed, preventing both self-inclusion and access to later items. Under the principal `online` policy, a validation target may retrieve earlier training flows; a test target may retrieve earlier training, validation, and test flows. Flows later in stable start-time order and all historical labels remain unavailable. This policy approximates a detector whose state persists through chronological deployment.

Context eligibility is defined by stable flow-start order: every context index occurs before its target index. The stronger condition $t_j^{\text{end}} \leq t_i^0$ is not enforced during Stage 2 export. A long-lived overlapping flow can therefore precede the target even when its complete embedding would not have been available at the target’s start time. What is evaluated is thus start-time-ordered, label-free context rather than a fully timed streaming system. Production eligibility should instead be based on flow completion or embedding availability.

The `time_only` relation denotes global chronological recency; it does not provide continuous inter-flow gaps to Stage 2. Explicit microsecond timing is confined to Stage 1, whereas Stage 2 represents relative age within the selected context.

Short sequences are padded on the left during collation, keeping the target at the final valid position:

$$[\text{PAD}, \dots, \text{PAD}, \mathbf{z}_{j_1}^{(1)}, \dots, \mathbf{z}_{j_{K_i}}^{(1)}, \mathbf{z}_i^{(1)}]. \quad (4.31)$$

A Boolean mask prevents the physical padding positions from affecting either positional encoding or attention.

4.3.3 Context-Age Positional Encoding

For a target with K_i historical tokens, age 0 is assigned to the target, age 1 to the most recent historical flow, and age K_i to the oldest retained item. Writing this index as $a_{i,k}$, Stage 2 adds the sinusoidal encoding

$$\text{PE}_{i,k,2j} = \sin\left(\frac{a_{i,k}}{10000^{2j/d}}\right), \quad (4.32)$$

$$\text{PE}_{i,k,2j+1} = \cos\left(\frac{a_{i,k}}{10000^{2j/d}}\right). \quad (4.33)$$

Because age is independent of absolute tensor position, additional left padding does not change it. Identical target-history orderings consequently receive identical encodings even when batch sequence lengths differ.

4.3.4 Target-Query Gated Multi-Head Attention

The principal Stage 2 model, `target_query_gated`, does not update every historical token through pairwise self-attention. Instead, the current flow supplies a single query over its eligible history. Let $\mathbf{c}_i$ denote the projected Stage 1 representation of the current flow and $\tilde{\mathbf{H}}_i \in \mathbb{R}^{K_i \times d}$ the age-encoded history. Attention head h computes

$$\mathbf{q}_i^h = W_Q^h \text{LN}(\mathbf{c}_i + \text{PE}(0)), \quad (4.34)$$

$$\mathbf{K}_i^h = \text{LN}(\tilde{\mathbf{H}}_i) W_K^h, \quad (4.35)$$

$$\mathbf{V}_i^h = \text{LN}(\tilde{\mathbf{H}}_i) W_V^h, \quad (4.36)$$

$$\mathbf{h}_i^h = \text{softmax}\left(\frac{\mathbf{q}_i^h (\mathbf{K}_i^h)^\top}{\sqrt{d_h}} + M_i\right) \mathbf{V}_i^h. \quad (4.37)$$

Concatenation and projection of the head outputs produce the historical summary $\mathbf{h}_i$. When the eligible history is empty, $\mathbf{h}_i$ is set explicitly to zero so that an artificial padding item cannot supply context.

Difference and multiplicative terms compare the current representation with its context summary:

$$\mathbf{r}_i = [\mathbf{c}_i; \mathbf{h}_i; \mathbf{c}_i - \mathbf{h}_i; \mathbf{c}_i \odot \mathbf{h}_i], \quad (4.38)$$

$$\mathbf{v}_i = [\mathbf{h}_i; \mathbf{c}_i - \mathbf{h}_i; \mathbf{c}_i \odot \mathbf{h}_i]. \quad (4.39)$$

A dimension-wise gate controls the subsequent context update:

$$\mathbf{g}_i = \sigma(G(\mathbf{r}_i)), \quad (4.40)$$

$$\mathbf{d}_i = U(\mathbf{v}_i), \quad (4.41)$$

$$\mathbf{f}_i = \text{LN}(\mathbf{c}_i + \mathbf{g}_i \odot \mathbf{d}_i). \quad (4.42)$$

If no history exists, $\mathbf{g}_i$ and $\mathbf{d}_i$ are both zero and the current-flow representation passes through unchanged. A residual feed-forward block then produces

$$\mathbf{z}_i^{(2)} = \text{LN}(\mathbf{f}_i + \text{FFN}(\mathbf{f}_i)), \quad (4.43)$$

$$\ell_i^{(2)} = h_2(\mathbf{z}_i^{(2)}). \quad (4.44)$$

Target-query attention preserves a direct path for the current flow and uses history as a learned correction. Since there is only one query, the attention-score matrix grows linearly rather than forming the $W \times W$ matrix required by full self-attention. Multi-head retrieval over a 128-flow window is therefore computationally practical.

4.4 Imbalance-Aware Objective and Stage Interface

4.4.1 Focal Objective and Controlled Sampling

Two-class softmax outputs and focal loss are used in both stages. For target y_i with assigned-class probability p_{i,y_i} ,

$$\mathcal{L}_{\text{focal}} = -\frac{1}{B} \sum_{i=1}^B \alpha_{y_i} (1 - p_{i,y_i})^\gamma \log p_{i,y_i}. \quad (4.45)$$

Originally introduced for severe foreground/background imbalance, focal loss down-weights examples that are already classified easily [42]. Table 5.2 reports the chosen focal exponent, class weights, and label-smoothing values.

A controlled weighted sampler additionally determines training batches. Given class counts n_0 and n_1 and desired expected positive share q , negative and positive examples receive

$$w_0 = \frac{1 - q}{n_0}, \quad w_1 = \frac{q}{n_1}. \quad (4.46)$$

Sampling changes expected minority exposure but leaves both the underlying dataset and the loss definition intact. Chapter 5 specifies stage-specific target proportions and loader policies.

4.4.2 Sequential Stage Interface

After Stage 1 has been trained for flow classification, its validation-selected checkpoint is restored. A deterministic loader without sampling exports one $\mathbf{z}_i^{(1)}$ and the required context metadata for every flow. These embeddings remain fixed during Stage 2 training, and no Stage 2 gradient reaches the earlier encoder. The sequential interface consequently allows Stage 1 comparisons to vary intra-flow representation while all Stage 2 models reason over a common representation.

5 Experimental Methodology

Within a comparison block, the data manifest, fitted preprocessing state, optimisation budget, checkpoint criterion, threshold rule, and random seed remain fixed. Validation PR-AUC determines the checkpoint, and validation class-1 F1 determines the decision threshold. The untouched test partition is evaluated only after both choices have been fixed. This protocol limits temporal leakage and feedback from test results, two sources of optimistic NIDS estimates [2, 40].

5.1 Evaluation Manifests and Leakage Controls

Evaluation is organised around two chronological manifests. Comparisons of Stage 1 architectures, encodings, fusion schemes, sequence lengths, and selection policies use 35,410 test flows: 34,003 negative and 1,407 positive. The hierarchical Scheme C reference and every Stage 2 experiment instead use the complete 84,380-flow test partition, comprising 80,860 negative and 3,520 positive flows. Results from these manifests are not paired, even when their rounded values happen to be close; effect sizes are calculated only between models sharing the same manifest.

All models within a block receive identical flows, packet and flow tensors, partition assignments, and train-fitted preprocessing. Training data alone determine category vocabularies, clipping limits, scaling parameters, and sampler weights. Configured class weights and sampling targets are also chosen without test feedback; neither validation nor test observations contribute to fitted transformations.

One frozen Stage 1 checkpoint supplies all Stage 2 inputs. All histories follow the start-time-ordered eligibility and split policy defined in Section 4.3.2. Embedding files, metadata ordering, context indices, and masks are reused across Stage 2 architectures, isolating the encoder or relation under study from context availability.

5.1.1 External Evaluation Protocol

External evaluation keeps `ar002_et12_20260511_001` as the source training dataset and does not merge either external dataset into the source chronological partitions. The source-trained Stage 1 and Stage 2 checkpoints are applied to compatible external-test artefacts. External labels are reserved for metric calculation. Feature order, training-fitted clipping and scaling parameters, categorical vocabularies, model checkpoints, context relation, context capacity, and the source-validation decision threshold remain unchanged. No external test label is used for checkpoint selection, early stopping, threshold selection, or model comparison.

For CICIDS2017, the external flows are kept in chronological order and divided into non-overlapping 20,000-flow windows. Before model-performance comparison, windows are screened by class composition: label 1 must remain less frequent than label 0, matching the qualitative imbalance direction of the source dataset. This prevalence criterion yields `w01`, `w02`, `w06`, `w07`, and `w08`; it is not defined from their predictive scores.

Stage 2 histories contain only flows earlier than the target within the permitted external evaluation stream; labels are never included. Each reported window retains its original class composition. Results are presented separately by window and after pooling the five

confusion matrices. Pooled ROC-AUC and PR-AUC require the concatenated flow-level scores to be recomputed. They cannot be inferred from confusion matrices or obtained by averaging window-level values.

The company-provided firewall capture is evaluated as one 38,513-flow cross-server set. Since it originates from a different server environment, it tests environmental transfer while remaining organisational data. External scores are not used to replace the internal chronological test: the internal test measures later source-domain traffic, whereas the two external evaluations measure public cross-dataset and company cross-server transfer.

5.2 Experimental Matrix

Table 5.1 distinguishes experiments on the packet encoder from those evaluating the complete hierarchy.

TABLE 5.1: Experimental matrix. Only the factor listed in the final column is changed within each experiment group.

Group	Stage	Fixed reference	Compared alternatives	Varied factor
S1-E1	1	Stage 1 manifest	Flow-statistics MLP; packet-sequence models; proposed Stage 1	Encoder family
S1-E2	1	Scheme B, $L = 128$, head	Both, position-only, time-only, no encoding	Temporal/order encoding
S1-E3	1	Both encodings, $L = 128$, head	Schemes A, B, C	Flow-statistics integration
S1-E4	1	Scheme B, head	$L = 8, 16, 32, 64, 128, 256$	Packet budget
S1-E5	1	Scheme B, $L = 128$	Head, head-tail, random	Truncation policy
S2-E1	2	Full Stage 2 manifest; same frozen embeddings	No-context MLP; Transformer; LSTM; GRU; CNN+LSTM; proposed	Context encoder
S2-E2	2	Proposed model, $W = 128$	Time-only, source-host, destination-host, endpoint	Context relation
S2-E3	2	Proposed model, source-host	$W = 16, 32, 64, 128, 256$	Context capacity

5.2.1 Stage 1 Baselines

Stage 1 is evaluated against the following baselines:

- **Flow-statistics MLP.** A multilayer perceptron consumes only the flow-statistics vector. It provides the direct reference for testing whether packet-level sequence information adds value beyond classification based on flow statistics.
- **CNN1D.** Parallel one-dimensional convolutions with kernel sizes 3, 5, and 7 capture local packet motifs, followed by global pooling and classification. A second variant concatenates the log inter-arrival feature to each packet record. The design follows the general use of multi-kernel convolution for sequence classification [43].
- **LSTM and BiLSTM.** Two-layer recurrent encoders with hidden width 128 and mean pooling provide unidirectional and bidirectional sequence references. The LSTM-with-time variant concatenates the stored log inter-arrival value to each packet vector [44].
- **GRU.** A two-layer, width-128 GRU with mean pooling tests whether a lighter gated recurrent encoder is sufficient [45].

Packet tensors, masks, the chronological partition, focal objective, sampler, optimiser, training budget, and model-selection protocol are shared by all Stage 1 baselines. The baseline architectures are not matched by parameter count; trainable parameters and runtime are instead reported beside predictive metrics to expose performance–efficiency trade-offs.

5.2.2 Stage 1 Ablations and Sensitivity Tests

Four variants separate the effects of order and elapsed time: (i) joint positional and time-aware encoding, (ii) position only, (iii) time only, and (iv) neither coordinate. Packet features and Transformer capacity are unchanged. This control matters because appending an interval to a feature vector is not equivalent to using elapsed time as the coordinate of a sinusoidal sequence representation.

The three schemes in Table 4.1 assess how flow statistics should be incorporated. Scheme A repeats the flow-statistics vector at every packet position; Scheme B removes it; Scheme C applies bounded token-FiLM conditioning from a separate branch. Packet-encoder and truncation experiments use Scheme B to isolate sequence modelling. Scheme C constitutes the complete Stage 1 model and provides embeddings for Stage 2.

Sequence-length sensitivity is tested at $L \in \{8, 16, 32, 64, 128, 256\}$ with `head` selection. A separate experiment holds $L = 128$ and contrasts `head`, `head_tail`, and deterministic random sampling. No other setting changes in either sweep.

5.2.3 Stage 2 Baselines and Context Ablations

Five controls are used to assess the target-query gated attention model:

- **No-context MLP** classifies only the current Stage 1 embedding and provides a minimal current-flow-only lower bound. It has substantially fewer parameters than the contextual encoders and therefore does not isolate the value of history.
- **Vanilla Transformer** applies full self-attention to the same context sequence and classifies its last valid token [12].
- **LSTM** and **GRU** replace attention with recurrent context encoders while preserving the Stage 1 inputs, context window, masks, and classifier protocol [44, 45].
- **CNN+LSTM** applies parallel temporal convolutions with kernel sizes 3, 5, and 7 before a two-layer LSTM, combining local pattern extraction with recurrent context modelling [43, 44].

During architecture comparison, each contextual baseline receives the same `source_host` histories at $W = 128$. The current flow occupies the final valid position, leaving room for at most 127 historical flows. The no-context control keeps the target embedding and discards the preceding history.

Relation sensitivity holds the proposed encoder and $W = 128$ fixed while varying eligibility among `time_only`, `source_host`, `destination_host`, and `endpoint`. Window sensitivity instead fixes `source_host` and evaluates $W \in \{16, 32, 64, 128, 256\}$. Run summaries contain predictive metrics and confusion matrices but do not retain target-level context occupancy or age. Interpretation of the window sweep is therefore limited by this missing diagnostic information.

5.3 Training, Model Selection, and Thresholding

5.3.1 Principal Training Configuration

Principal architecture and optimisation settings are listed in Table 5.2. Both stages use AdamW, which decouples weight decay from adaptive gradient updates [46], together with focal loss to downweight easy majority-class examples [42].

TABLE 5.2: Principal optimisation configuration. Values changed in a named ablation are the only exceptions.

Setting	Stage 1	Stage 2
Architecture and input	Scheme C; $L = 128$; head selection; $d = 128$	Target-query gated attention; source-host; $W = 128$; $d = 128$
Batch size	128	128
Optimiser	AdamW	AdamW
Learning rate	1×10^{-4}	3×10^{-5}
Weight decay	1×10^{-4}	3×10^{-4}
Loss	Focal, $\gamma = 1$, $\alpha = [1, 1.25]$, no label smoothing	Focal, $\gamma = 1$, $\alpha = [1, 1.01]$, no label smoothing
Sampler target	10% class-1 examples	5% class-1 examples
Maximum epochs	500	500
Early-stopping patience	25 epochs	15 epochs
Checkpoint criterion	Validation PR-AUC	Validation PR-AUC

Weighted sampling with replacement is restricted to training mini-batches. Validation, test, and embedding-export loaders preserve exactly one row per flow and do not sample, preventing duplicated embeddings or changes to the empirical evaluation distribution. CUDA runs use automatic mixed precision. Stage 1 lowers its learning rate after a plateau in validation PR-AUC, whereas Stage 2 follows five warm-up epochs with cosine annealing.

All reported configurations use one fixed random seed. Run-to-run variation is therefore not estimated, so small differences between configurations are interpreted descriptively rather than as statistically significant [47, 48].

5.3.2 Checkpoint and Decision-Threshold Selection

Checkpoint and decision-threshold selection are separate operations applied uniformly across experiments. First, the checkpoint with the highest validation PR-AUC is restored. Validation predictions from that checkpoint then determine the class-1 threshold:

$$\delta^* = \arg \max_{\delta \in \Delta} F_{1,1}^{\text{val}}(\delta). \quad (5.1)$$

In Stage 1, Δ contains thresholds induced by the validation precision–recall curve; Stage 2 searches 199 equally spaced values between 0.01 and 0.99. Once selected, δ^* is fixed and transferred to the test partition without adjustment. Test observations play no role in checkpoint, threshold, or hyperparameter selection.

5.4 Evaluation Metrics

Metric calculations follow scikit-learn definitions [49]. Label 1 is the alert-derived positive class, and label 0 is the negative class. Confusion matrices use the fixed orientation

$$\mathbf{C} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix}, \quad (5.2)$$

with true labels on rows and predictions on columns. Fixing this convention removes ambiguity when matrices are transferred between software packages.

5.4.1 Class-Wise and Macro-Averaged Measures

For $c \in \{0, 1\}$, class-wise precision, recall, and F1 are

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad (5.3)$$

$$R_c = \frac{TP_c}{TP_c + FN_c}, \quad (5.4)$$

$$F_{1,c} = \frac{2P_cR_c}{P_c + R_c}. \quad (5.5)$$

Undefined ratios take the value zero, consistent with the implementation's `zero_division=0` setting. The macro-averaged metrics are

$$P_{\text{macro}} = \frac{P_0 + P_1}{2}, \quad (5.6)$$

$$R_{\text{macro}} = \frac{R_0 + R_1}{2}, \quad (5.7)$$

$$F_{1,\text{macro}} = \frac{F_{1,0} + F_{1,1}}{2}. \quad (5.8)$$

Macro-F1 is the primary F1 measure and is abbreviated to “F1” in high-level tables. It assigns equal weight to the negative and positive classes despite their unequal prevalence; macro-precision and macro-recall are interpreted analogously. To reveal error changes hidden by macro averaging, P_1 , R_1 , and $F_{1,1}$ are reported alongside these summaries.

Accuracy and weighted averages remain secondary: an always-negative rule would score approximately 96.055% accuracy on the raw data while detecting no positive flow. Neither high accuracy nor weighted F1 alone is therefore sufficient evidence of effective intrusion detection.

5.4.2 False-Positive Rate and Operational Error Counts

False-positive rate is defined as

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \quad (5.9)$$

and false-negative rate as

$$\text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}} = 1 - R_1. \quad (5.10)$$

Even a low FPR can generate substantial alert volume in a high-throughput network. It is therefore considered together with absolute FP and FN counts. At the evaluated operating point, a test partition with N_0 negative flows contains $N_0 \times \text{FPR}$ false positives.

5.4.3 Threshold-Independent Ranking Measures

ROC-AUC measures ranking through the true-positive rate over the range of false-positive rates [50]. Under strong class imbalance, however, a large negative population can co-exist with a high ROC-AUC. Average precision (AP) instead summarises precision–recall performance for the positive class and consequently receives greater emphasis here [39].

The implementation uses scikit-learn's `average_precision_score`, rather than trapezoidal integration under linearly interpolated precision–recall points:

$$\text{AP} = \sum_k (R_k - R_{k-1}) P_k, \quad (5.11)$$

where P_k and R_k are evaluated at successive score thresholds. Because random ranking has an expected AP near positive prevalence, observed values should be interpreted against the class-1 share of the relevant test manifest.

5.5 Efficiency and Resource Measurement

Experiments ran in Google Colaboratory Pro+ on an NVIDIA L4 GPU with 24 GB memory, using a company-funded allocation of 600 compute units [51, 52]. Those units regulate access to hosted resources and do not quantify a fixed amount of GPU computation. Runtime is compared only within the same stage because Stage 1 and Stage 2 differ in data loading, early stopping, and execution path.

Trainable parameters are counted directly from tensors that require gradients. Stage 1 wall time includes fitting, validation, and checkpointing. It is treated as a descriptive single-run cost because early stopping and variation in the hosted environment affect duration. For Stage 2 timing, gradients are disabled and the model is placed in evaluation mode. Per-flow latency is calculated as

$$t_{\text{flow}} = \frac{T_{\text{test}}}{N_{\text{test}}}, \quad Q = \frac{N_{\text{test}}}{T_{\text{test}}}, \quad (5.12)$$

where Q denotes flows processed per second. Timing starts after Stage 1 embeddings and context indices have been prepared, so it measures model evaluation rather than the complete path from PCAP to alert.

TABLE 5.3: Execution environment for the reported experiment blocks.

Component	Recorded environment
Hosted platform	Google Colaboratory Pro+
Purchased allocation	600 purchased compute units
Accelerator	NVIDIA L4 (24 GB)
Host processor and RAM	Google Colab hosted CPU; exact model not retained in the run metadata
Operating environment	Google Colab hosted Linux runtime
Learning software	PyTorch and scikit-learn on CUDA; exact version snapshot not retained
Traffic extractor	Suricata with the custom output module; exact version and revision not retained

Available run metadata do not retain the exact CPU, software snapshot, Suricata version, ruleset snapshot, configuration, custom-module revision, or PCAP checksum. Exact reproduction of runtime and label generation is consequently limited, and the missing provenance is identified as unavailable rather than reconstructed after the fact.

6 Results and Discussion

Unless explicitly qualified, F1, precision, and recall refer to macro averages; class-1 quantities carry a separate label. Discussion centres on macro-F1, PR-AUC, class-1 performance, FP, FN, and FPR because the majority class dominates accuracy. Numerical differences are interpreted only when the models share a test manifest.

6.1 Stage 1: Intra-Flow Representation

6.1.1 Architecture and Temporal-Encoding Effects

Stage 1 model families and encoding ablations are contrasted on a common flow test set in Figure 6.1; their parameter counts and training times appear in Table 6.1.

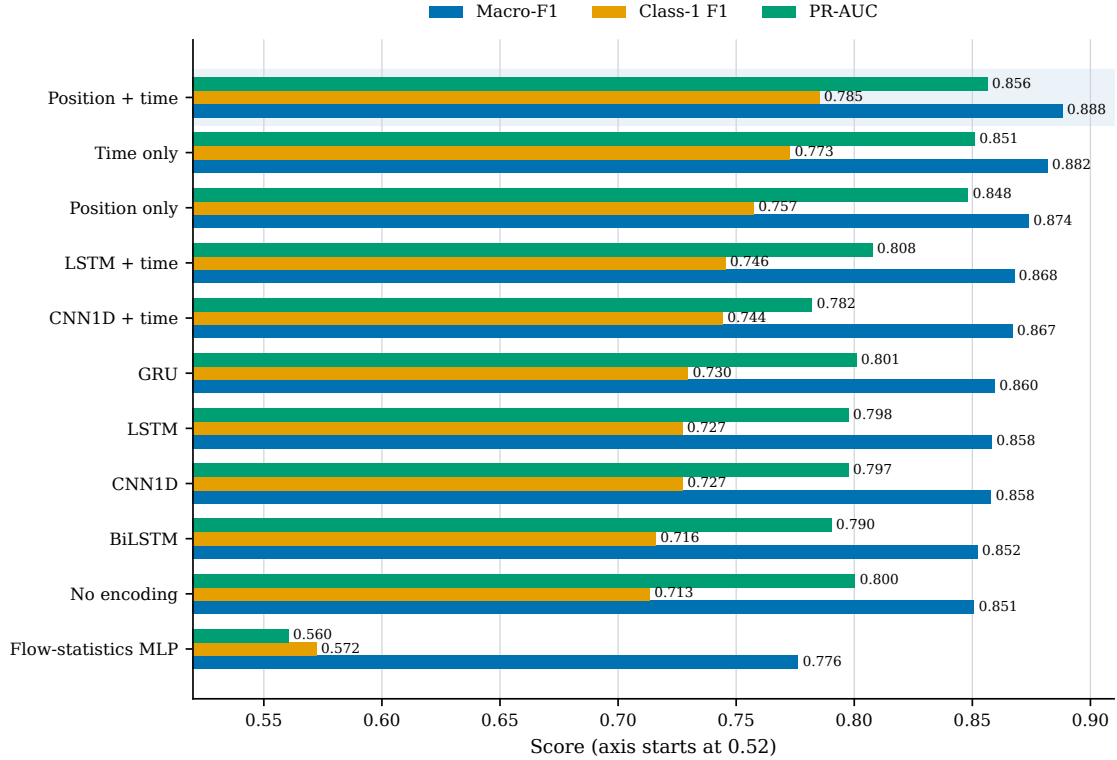


FIGURE 6.1: Stage 1 architecture and encoding comparison on the common flow test manifest. The horizontal axis begins at 0.52, and absolute values are printed at the end of each bar.

Among these models, position+time gives the Transformer the best overall balance: macro-F1 reaches 0.8882 and PR-AUC reaches 0.8564. LSTM+time is the strongest non-Transformer alternative by macro-F1, yet the Transformer improves this score by 2.05 percentage points. Relative to LSTM+time at the selected operating point, it recovers 95 additional positive flows at the cost of 10 more false positives. The gain therefore reflects a different error balance rather than a change in a summary score alone.

Under the evaluated encodings, time-only exceeds position-only, and combining them adds another 0.64 percentage points of macro-F1. Both FP and FN are also lower than under no encoding.

Ordinal position and cumulative time encode different quantities: the former records where a packet occurs, whereas the latter distinguishes a rapid burst from an identical order spread over a longer interval. The observed joint improvement is consistent with these encodings contributing complementary cues. It does not, however, identify which packet patterns account for the gain.

The lowest FPR, 0.8293%, belongs to the GRU, but that operating point misses 437 positive flows. Position+time introduces only 14 additional FP and removes 131 FN relative to the GRU. In this comparison, the lower FPR is accompanied by substantially more missed positives, so FPR should not be interpreted in isolation.

The flow-statistics MLP provides the direct flow-statistics-only control. It obtains macro-F1 0.7762 and PR-AUC 0.5604 on the same test composition, compared with 0.8882 and 0.8564 for position+time. Its 853 FP and 501 FN decrease to 296 and 306, respectively, for the packet-sequence model.

A cautious interpretation is that the flow-statistics summaries preserve some discriminatory signal but do not match the evaluated packet-sequence representation. The comparison does not reveal which omitted order or timing patterns account for the difference.

Operating-point errors are shown in Figure 6.2. Because the negative examples are shared, FP bar height is directly proportional to FPR.

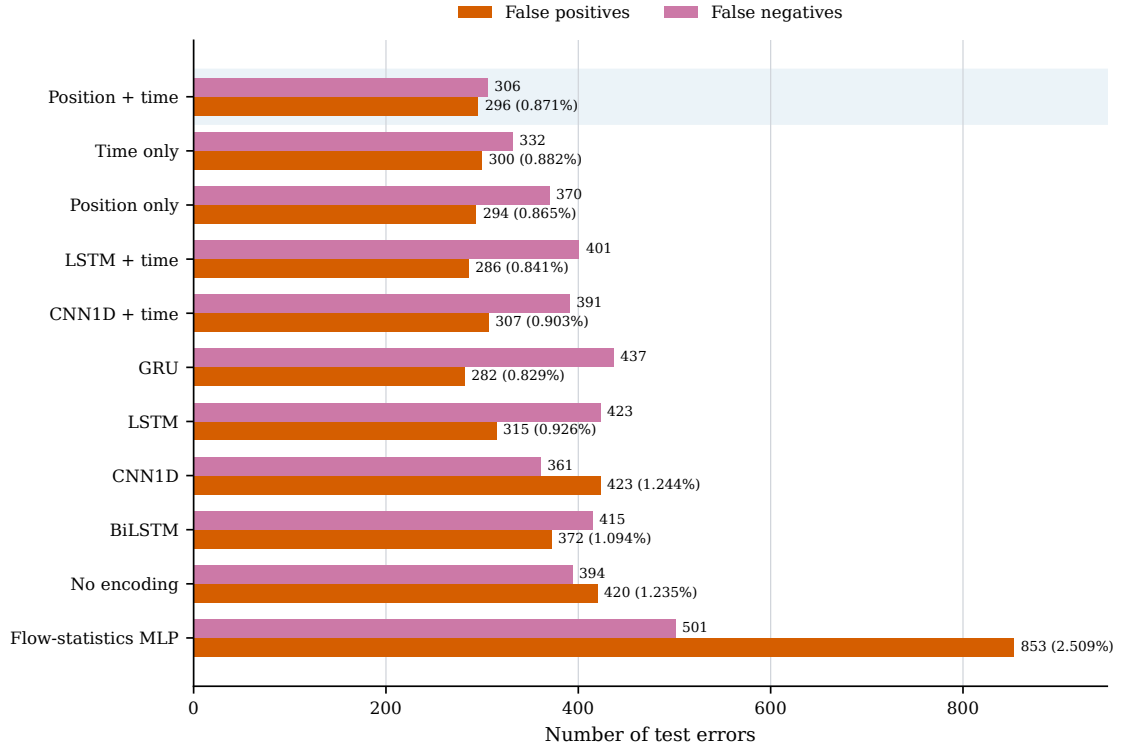


FIGURE 6.2: Stage 1 false-positive and false-negative counts on the common flow test manifest. FPR is printed beside each FP count.

For LSTM, adding explicit time coincides with reductions in both error types. For CNN, however, the result is mixed: its selected operating-point score improves, while PR-

AUC declines and FN rises. Thresholded F1 characterises one decision point, whereas PR-AUC evaluates ranking over thresholds.

Taken together, these results suggest that appending a timing feature does not produce consistent gains across the evaluated encoders. The way in which timing is represented within an encoder may matter, although the present comparison does not isolate the mechanism behind the architecture-specific differences.

TABLE 6.1: Stage 1 parameter counts and single-run training times measured on an NVIDIA L4 GPU in Google Colab.

Model	Trainable parameters	Training time (s)	Training time (min)
Position+time Transformer	344,963	4,337.6	72.29
Time-only Transformer	344,963	2,250.5	37.51
Position-only Transformer	344,963	1,994.7	33.25
LSTM+time	217,346	3,556.3	59.27
CNN1D+time	70,274	1,162.4	19.37
GRU	162,690	3,684.4	61.41
LSTM	216,834	3,381.2	56.35
CNN1D	68,354	1,591.0	26.52
BiLSTM	564,738	2,625.7	43.76
No-encoding Transformer	344,963	2,213.9	36.90

No MLP efficiency entry is shown in Table 6.1, as its parameter count and training time were not retained.

CNN1D+time records the shortest Stage 1 training time, 1,162.4 seconds; position+time requires 4,337.6 seconds. Despite the BiLSTM’s larger parameter count, the proposed encoder has the longest measured training run. Each value was collected on an NVIDIA L4 GPU, but it includes early stopping, data loading, and variation in the hosted runtime. These measurements describe total training cost, not architecture-only throughput.

6.1.2 Packet and Flow-Statistics Integration

The fusion comparison in Figure 6.3 uses one shared flow manifest. Scheme A tiles the flow-statistics vector across packet tokens, Scheme B retains packet inputs alone, and Scheme C conditions packet representations through bounded token-FiLM from a separate branch.

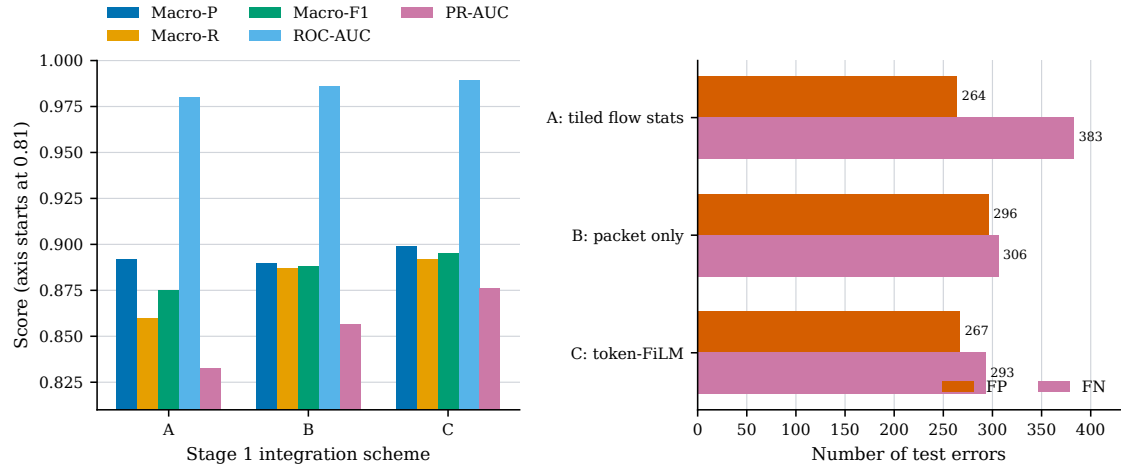


FIGURE 6.3: Controlled Stage 1 integration comparison on the common flow manifest. Left: macro metrics and threshold-independent ranking metrics. Right: false-positive and false-negative counts. The score axis is truncated and explicitly labelled.

Removing the tiled vector in Scheme B raises macro-F1 by 1.30 percentage points and PR-AUC by 2.38 points over Scheme A, although Scheme A retains a small macro-precision advantage. Scheme C is strongest overall, with macro-F1 0.8954, PR-AUC 0.8763, 267 FP, and 293 FN. Both error counts are lower than under packet-only Scheme B.

A plausible account of the Scheme A result is that repeating one flow-statistics vector at every position interferes with distinctions among packet tokens. Scheme C’s performance is also consistent with flow statistics being more effective when encoded through a dedicated pathway than when copied into each packet position. Neither mechanism is verified by this ablation. Scheme C introduces extra parameters as well as a separate flow-statistics pathway, so attribution must remain at the level of the integrated design.

6.1.3 Sequence Length and Packet Selection

Figure 6.4 presents both sensitivity analyses: sequence length is traced in the line panel, and selection policies are compared through grouped precision and recall bars.

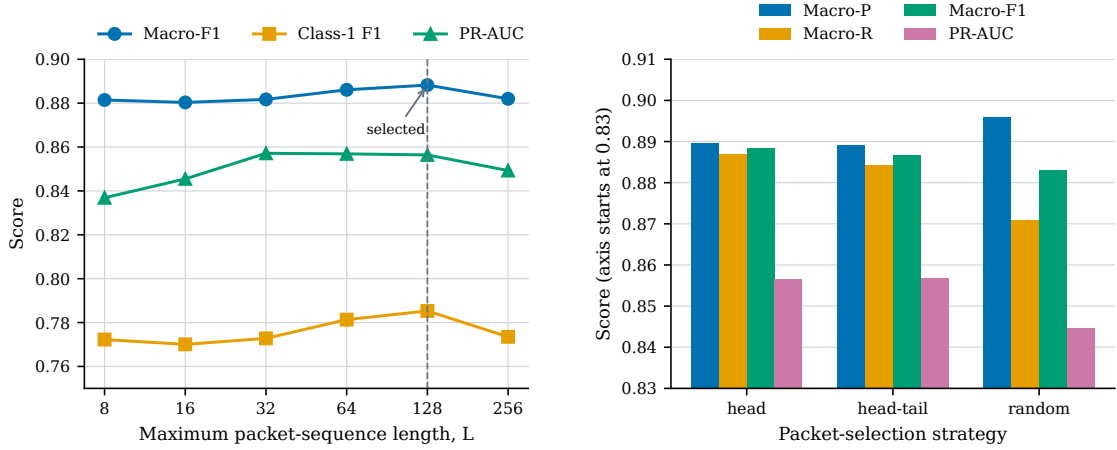


FIGURE 6.4: Stage 1 sensitivity analysis. Left: maximum packet-sequence length under head selection. Right: packet-selection strategy at $L = 128$. Dashed lines identify the selected sequence length. Truncated axes are explicitly labelled.

Longer sequences do not produce monotonic improvement. Ranking metrics are lower at the shorter tested budgets. Among the tested values, $L = 128$ gives the highest macro-F1 and the strongest overall error balance. Increasing the budget to 256 adds 46 FP and lowers both macro-F1 and PR-AUC. Thus, the additional packet budget does not improve predictive performance in this experiment.

The aggregate sweep does not identify why the curve peaks at $L = 128$. Later packets might dilute cues concentrated near the beginning of a flow, but flow-length composition, truncation coverage, or optimisation effects could produce the same pattern. Distinguishing among these accounts would require length-stratified evaluation. Consequently, $L = 128$ is treated as the best observed budget for this capture, not evidence of a universal sequence horizon.

At $L = 128$, the macro-F1 difference between head and head-tail is only 0.16 percentage points. Under the single-seed design described in Section 5.3.1, this margin is treated as practically inconclusive. Random selection shifts the detector towards lower FPR and more FN. Head remains the principal policy because it is deterministic, provides the best observed macro-F1 and class-1 recall, and preserves a contiguous early prefix without waiting for the flow’s final packets. Its small advantage over head-tail is not interpreted as a robust ordering.

6.2 Stage 2: Inter-Flow Context

6.2.1 Gain over Independent Flow Classification

The principal cross-stage comparison appears in Table 6.2. Both rows share the same test flows: one classifies each Scheme C embedding independently, and the other augments that frozen representation with start-time-ordered source-host history.

TABLE 6.2: Independent Stage 1 classification and the complete hierarchical system on the common full-data manifest.

Model	P_{macro}	R_{macro}	$F_{1,\text{macro}}$	$F_{1,1}$	ROC-AUC	PR-AUC	FPR (%)
Stage 1 Scheme C	0.8990	0.8919	0.8954	0.7994	0.9890	0.8763	0.8249
Stage 2 source-host	0.9328	0.9224	0.9275	0.8610	0.9938	0.9345	0.5429

Adding Stage 2 raises macro-F1 by 3.21 percentage points and PR-AUC by 5.82 points relative to independent Scheme C classification. The contextual system removes 228 FP and 205 FN, a relative FPR reduction of approximately 34.2%. Since both error types decline, the gain is not an accuracy artefact or a simple shift towards one class.

This comparison concerns the hierarchy as an integrated system and does not by itself identify the source of the improvement. The capacity-control qualification in Section 5.2.3 therefore applies to this result.

6.2.2 Which Relation Defines Useful Context?

Figure 6.5 links ranking quality to operating behaviour. Macro-F1 and PR-AUC are shown on the left; class-1 recall and FPR locate each relation on the right.

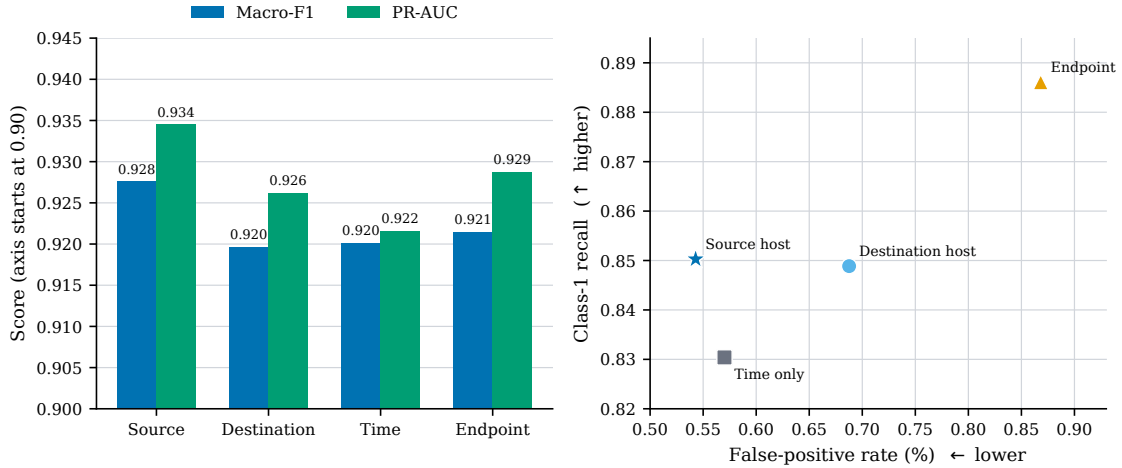


FIGURE 6.5: Stage 2 context-relation ablation at $W = 128$. Left: macro-F1 and PR-AUC. Right: class-1 recall versus FPR, where the desirable region is the upper-left corner.

Source-host history yields the strongest balance: macro-F1 is 0.9275, PR-AUC 0.9345, and FPR is the lowest among the relations. Endpoint context misses fewer positives but produces more false alerts, while time-only context ranks flows less effectively and produces more FN.

This error trade-off has an operational implication. Endpoint context may be preferable when false negatives carry a higher cost; under the balanced criterion used here, source-host remains the selected relation.

Repeated outbound activity from one initiator may preserve evidence of scanning, retries, or coordinated connections. Under this interpretation, global recency may mix unrelated flows. Destination history may combine many benign clients of a widely used service, while endpoint matching may admit a more heterogeneous neighbourhood that

also retrieves lateral or bidirectional activity. This account is broadly consistent with prior relational NIDS research [27, 30], but it remains unverified because the binary labels do not identify the attack families responsible for the observed differences.

6.2.3 How Much Context Is Useful?

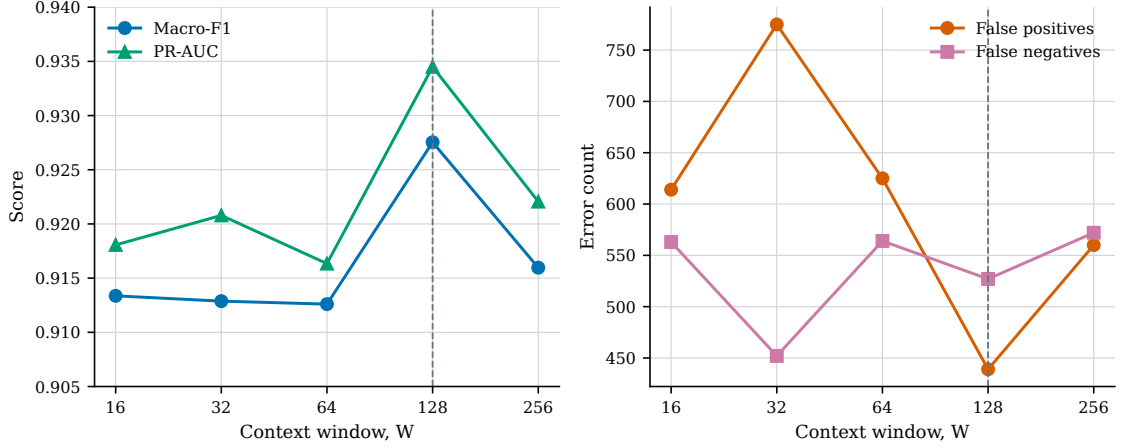


FIGURE 6.6: Source-host context-window sensitivity. Left: macro-F1 and PR-AUC. Right: false-positive and false-negative counts. The dashed line marks the selected window $W = 128$.

Context capacity is also non-monotonic in Figure 6.6. The $W = 32$ setting maximises macro-recall, but it also produces 775 FP. At $W = 128$, macro-F1 reaches 0.9275, PR-AUC reaches 0.9345, and FPR falls to its minimum of 0.5429%, giving the strongest joint outcome. Both FP and FN rise at $W = 256$.

The non-monotonic pattern could reflect a finite useful context horizon: short histories may omit relevant activity, whereas increasingly old flows from the same source may be less relevant to the current event. The aggregate results cannot confirm this account. The run summaries do not retain target-level counts or age distributions for valid context items, so older evidence, padding frequency, and unequal history coverage cannot be disentangled. Performance stratified by realised context length and elapsed span would be needed to test the horizon hypothesis directly. Within the present design, $W = 128$ is therefore an empirical operating point for this capture, not a universal optimum.

6.2.4 Stage 2 Architecture Comparison

Architecture comparison fixes the Stage 1 embeddings, source-host relation, $W = 128$, online policy, objective, optimiser, sampler, checkpoint criterion, and validation threshold search. Only the context encoder and dimensions specific to that encoder vary. Six architectures are displayed in Figure 6.7; FP and FN are shown separately in the second panel so that aggregate F1 does not conceal their trade-off.

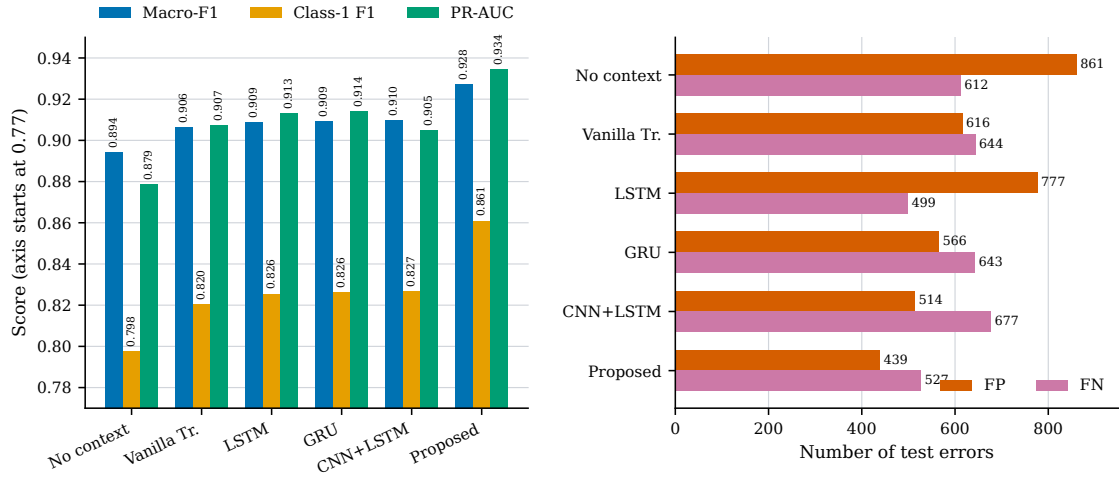


FIGURE 6.7: Stage 2 architecture comparison on the common flow test manifest. Left: macro-F1, class-1 F1, and PR-AUC. Right: threshold-dependent false-positive and false-negative counts. Truncated score axes are labelled.

Target-query attention produces the best combined result, with macro-F1 0.9275, class-1 F1 0.8610, and PR-AUC 0.9345. Among the baselines, CNN+LSTM records the closest macro-F1 at 0.9097, while GRU records the highest PR-AUC at 0.9141. LSTM has slightly higher macro-recall, yet that advantage is accompanied by weaker threshold-independent ranking and a less favourable false-alert balance.

Confusion matrices make these differences operationally concrete. Against the vanilla Transformer, target-query attention eliminates 177 FP and 117 FN on the same flows. LSTM recovers 28 more positives than the proposed model but generates 338 additional false alerts. The macro-F1 gain over vanilla attention is therefore not obtained by exchanging one error type for the other, and the observed precision advantage over the highest-recall recurrent baseline is substantial.

The target-query design may focus the contextual summary more directly on the current flow. Vanilla self-attention updates the full context sequence and classifies its final valid state, whereas the proposed encoder keeps the current flow as the query and gates the retrieved historical summary. The simultaneous improvement in macro-F1 and PR-AUC is consistent with this intended inductive bias, but does not verify the explanation. Other architectural choices and parameter counts also differ, so the comparison does not cleanly isolate the query pattern.

6.2.5 Efficiency and Operational Trade-Offs

Parameter count and measured evaluation latency are related in Figure 6.8, with macro-F1 annotated for each architecture. A logarithmic parameter axis accommodates the range from 514 to more than 1.2 million parameters.

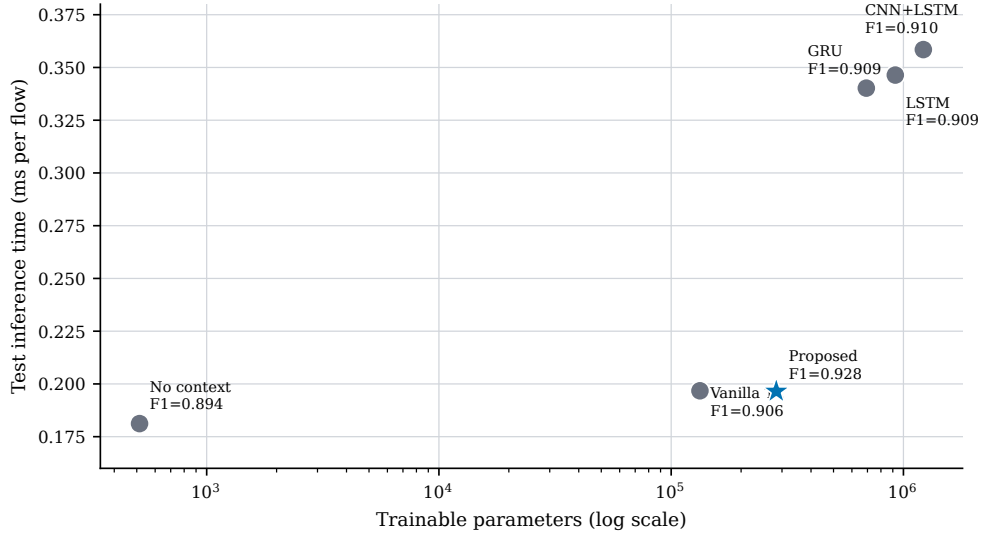


FIGURE 6.8: Stage 2 performance–efficiency trade-off using the model-level latency defined in Section 5.5.

All profiles in Figure 6.8 were collected on an NVIDIA L4. The proposed model has 283,522 parameters and evaluates one flow in 0.1965 ms on average, equivalent to approximately 5,089 flows per second for the measured model path. Vanilla Transformer latency is 0.1967 ms. A difference of 0.0002 ms is too small to support a speed claim; the meaningful observation is latency parity accompanied by a better error balance.

Compared with GRU, LSTM, and CNN+LSTM, parameter count falls by 59.0%, 69.3%, and 76.7%, respectively; measured latency falls by 42.2%, 43.3%, and 45.2%. Unlike the negligible margin over vanilla attention, these differences show lower parameter counts and model-level latency than the evaluated recurrent alternatives. The measurement scope and exclusions are defined in Section 5.5.

6.3 External Generalisation

The source models are trained on `ar002_et12_20260511_001`. External evaluation then considers the public CICIDS2017 Wednesday working-hours capture and the company-provided `firewall_test_20250731_160700` capture from a different server. Table 6.3 reports each chronological CICIDS2017 window separately so that temporal heterogeneity is not hidden by one aggregate score.

TABLE 6.3: External evaluation on prevalence-compatible public CICIDS2017 Wednesday windows and a separate company server capture.

Dataset/window	Flows	Positive (%)	Accuracy	$F_{1,\text{macro}}$	$F_{1,1}$	ROC-AUC	PR-AUC
CICIDS2017 w01	20,000	18.195	0.9478	0.9125	0.8569	0.9816	0.9088
CICIDS2017 w02	20,000	13.515	0.9636	0.9226	0.8663	0.9728	0.9123
CICIDS2017 w06	20,000	16.480	0.9300	0.8536	0.7479	0.9758	0.8906
CICIDS2017 w07	20,000	13.340	0.9358	0.8640	0.7651	0.9691	0.8419
CICIDS2017 w08	20,000	15.235	0.9583	0.9220	0.8688	0.9884	0.9397
Company cross-server	38,513	3.285	0.9882	0.9026	0.8112	0.9870	0.8771

Across the five public windows selected by the minority-positive prevalence criterion, the arithmetic mean of window-level macro-F1 is 0.8949, with a range from 0.8536 to 0.9226. Pooling their confusion matrices gives 82,292 TN, 2,355 FP, 2,937 FN, and 12,416 TP over 100,000 flows. The resulting pooled macro-F1 is 0.8966, class-1 F1 is 0.8243, and FPR is 2.7821%. This pooled macro-F1 is not identical to the mean window-level score because the former is recomputed after aggregating counts. A pooled PR-AUC or ROC-AUC is not claimed here because those ranking measures require concatenated flow-level probabilities rather than confusion-matrix counts.

TABLE 6.4: External-test confusion matrices and operating-point error rates.

Dataset/window	TN	FP	FN	TP	FPR (%)
CICIDS2017 w01	15,830	531	513	3,126	3.2468
CICIDS2017 w02	16,914	383	345	2,358	2.2143
CICIDS2017 w06	16,521	183	1,218	2,078	1.0955
CICIDS2017 w07	16,622	710	575	2,093	4.0965
CICIDS2017 w08	16,405	548	286	2,761	3.2325
CICIDS2017 pooled	82,292	2,355	2,937	12,416	2.7821
Company cross-server	37,087	161	292	973	0.4322

Performance varies across the five public windows in which label 1 remains the minority. Windows **w01** and **w02** provide balanced class-1 precision and recall, while **w08** gives the strongest class-1 F1 and PR-AUC. Window **w06** has class-1 precision of 0.9191, but recall falls to 0.6305 and 1,218 positive flows are missed; its ROC-AUC and PR-AUC nevertheless remain 0.9758 and 0.8906, respectively. Window **w07** has the largest FP count and the lowest PR-AUC.

The combination of strong ranking metrics and low recall in **w06** may indicate a calibration shift. Temporal change could have altered the probability scale or the suitability of the source-domain threshold without eliminating ranking ability. Confirming this interpretation would require calibration analysis and threshold re-evaluation on an independent validation set. By contrast, **w07**’s lower PR-AUC may reflect weaker class separation in addition to its unfavourable operating-point error balance.

On the separate company server, macro-F1 reaches 0.9026, class-1 F1 reaches 0.8112, and PR-AUC reaches 0.8771. The FPR remains low at 0.4322%, but class-1 recall is 0.7692, corresponding to 292 FN among 1,265 positive flows. Relative to the source-capture result, the error difference is expressed primarily through missed positives rather than an increase in false alerts.

Together, the public and organisational tests provide preliminary evidence that useful discrimination persists outside the source capture. The window-level variation cautions against treating external performance as invariant across time or environment. Moreover, as discussed in Section 3.2, these comparisons do not isolate dataset shift to one factor. The evidence therefore supports external generalisation only under the implemented evaluation pipeline, not universal transfer to arbitrary networks.

6.4 Cross-Stage Discussion

The Stage 1 ablations show higher performance for the position+time packet encoder than for the flow-statistics MLP. In the Stage 2 comparison, adding start-time-ordered source-

host history to a fixed Stage 1 representation lowers both FP and FN.

Viewed together, these observations support a two-level interpretation: Stage 1 represents order and elapsed time that are absent from the flow-statistics vector, while Stage 2 supplies historical evidence that is unavailable to independent flow classification. In this respect, the hierarchy extends packet-aware systems such as GTID and modular flow encoders such as FlowTransformer with an explicitly evaluated cross-flow reasoning layer [14, 15].

With respect to RQ1, time-aware packet modelling exceeds the flow-statistics MLP on a shared manifest, while the complete Scheme C configuration exceeds the tested tiled and packet-only alternatives. The sequence-length sweep adds a qualification: increasing the packet budget beyond $L = 128$ does not improve performance in this capture. For RQ2, Stage 2 lowers both FP and FN relative to independent Scheme C predictions. The relation and window experiments further show that performance varies with the context definition and is non-monotonic in window size.

Within the evaluated setting, these results suggest that context selection should be treated as a tunable design choice. They do not establish a universally optimal relation or horizon.

These findings accord with temporal sequence models and graph-based analyses that treat traffic records as dependent [18, 27, 28]. External evaluation provides preliminary evidence that useful signal persists across a public benchmark and a different company server, although its strength and error balance vary by environment and chronological window. The results do not establish source-host history as the best relation for every attack, nor can binary labels reveal which families benefit. A stronger cross-stage analysis would align individual errors, identify which Stage 1 FP and FN are corrected by Stage 2, and relate those transitions to relation type, realised context size, and context age.

Taken together, the observed improvements support treating intra-flow temporal structure and inter-flow relation selection as complementary design dimensions. The ablations do not, however, establish the exact mechanisms behind their gains.

7 Conclusion and Future Work

7.1 Conclusions

RQ1: intra-flow representation. Under the tested configurations, position+time encoding exceeds the individual and no-encoding variants, and the complete Scheme C configuration achieves the strongest Stage 1 result. The sequence-length experiments further show that observing more packets does not necessarily lead to better performance.

Within the tested setting, these results support an ordered, time-aware packet representation over reliance on flow statistics alone. They are consistent with complementary positional, temporal, and flow-statistics cues, but do not isolate the precise mechanism responsible for Scheme C’s gain.

RQ2: cross-flow context. For RQ2, the complete contextual system outperforms the evaluated independent-flow configuration, with both FP and FN lower. Among the tested relations, source-host context provides the strongest overall balance, whereas endpoint context shifts the operating point towards higher recall at the cost of more false positives. Across the evaluated Stage 2 architectures, the proposed target-query model achieves the strongest overall performance. This conclusion remains subject to the capacity-control qualification in Section 5.2.3.

Together, these findings support the interpretation that intra-flow temporal structure and selective inter-flow context contribute complementary evidence at different levels of the detection hierarchy. Temporal structure describes how a flow develops internally, while selective historical context relates that representation to earlier network activity.

In the observed comparisons, the complete two-stage configuration exceeds the evaluated independent Stage 1 and contextual baseline configurations on the source capture. The external tests provide preliminary evidence that useful discrimination persists. Pooled macro-F1 is 0.8966 across the evaluated CICIDS2017 windows, while cross-server macro-F1 is 0.9026 on the separate company server. Variation across the external windows indicates that transfer is non-uniform.

The main contribution of this thesis is therefore not the use of a Transformer architecture in isolation, but the design and evaluation of a two-stage hierarchical decomposition that explicitly combines these two levels of evidence.

7.2 Limitations and Scope

The dataset, label-provenance, and external-scope limitations established in Section 3.13 remain applicable. The single-seed qualification and the context-eligibility boundary are defined in Sections 5.3.1 and 4.3.2, respectively. The binary targets also cannot reveal which attack families benefit from each relation.

Freezing Stage 1 clarifies experimental attribution but gives up end-to-end optimisation. Reported Stage 2 latency is model-level rather than end-to-end, as defined in Section 5.5; deployment memory, tail latency, and concept drift remain unmeasured.

7.3 Future Work

Two methodological directions follow directly from these limitations: resource-efficient Stage 1 pre-training and relation-aware Stage 2 context that adapts to attack behaviour. Either extension should be accompanied by stronger statistical and external validation.

7.3.1 Resource-Efficient Self-Supervised Pre-Training

Stage 1 currently learns only from supervised binary targets, and large-scale pre-training was outside the available computational budget. The present experiments therefore do not show whether broader unlabelled traffic could yield a more reusable representation. Future work should pre-train the encoder on a wider collection of packet and flow records before task-specific fine-tuning. Such self-supervision may recover reusable traffic structure without plaintext payloads or labels for every flow [23, 4].

Several objectives fit the current feature space. Masked packet-feature modelling could reconstruct selected header, direction, length, and timing attributes. Predictive tasks could estimate a subsequent packet’s direction, discretised length, or log inter-arrival time. Temporal-order verification would distinguish authentic packet order from perturbations, while contrastive learning could align valid subsequences or augmented views of one flow. Raw addresses, ports, and similar identity fields should be removed or carefully randomised so that pre-training captures behaviour rather than host identity.

Pre-training relocates computation to an offline phase rather than removing it. Mixed precision, gradient accumulation, moderate sequence lengths, shared parameters, and progressive scaling of traffic samples should therefore be investigated. Under equal labelled-data budgets, downstream comparisons should cover random initialisation, linear probing of a frozen encoder, partial fine-tuning, and full fine-tuning. Beyond in-domain macro-F1, evaluation should ask whether pre-training improves label efficiency, rare-attack detection, convergence, and transfer to later or external captures.

7.3.2 Attack-Adaptive Relation-Aware Context Modelling

Stage 2 performance varies across the evaluated relation definitions. Among the tested relations, source-host history gives the best overall balance, whereas endpoint history exchanges additional false positives for higher attack recall.

A working hypothesis is that different attack mechanisms favour different relations: scanning may involve one source and many destinations, DDoS many sources and one destination, brute force repeated service interactions, and lateral movement a temporally connected endpoint chain. Attack-family labels are required to test this hypothesis. If it is supported, a future context selector could adapt to the target and hypothesised behaviour instead of using one globally fixed relation.

One extension is a multi-relation context pool containing source-host, destination-host, endpoint, protocol, service, subnet, and temporal-neighbour relations. Relation identity could enter through learned embeddings or attention biases; alternatively, separate key-value projections could be combined by a target-conditioned gate. In both cases, eligibility should be limited to embeddings available at decision time, enforced through flow-end or embedding-availability timestamps rather than start order alone. The horizon might also adapt to traffic density by selecting between duration-based and flow-count windows.

Testing this proposal requires attack-family labels or a defensible behavioural grouping beyond the binary target. Per-family recall, FPR, confusion patterns, and relation weights could help assess whether attacks draw on different contexts. Appropriate controls

include the current single relation, relation-agnostic attention, and parameter-matched multi-relation models. The present CICIDS2017 and cross-server tests should be extended to additional days, public datasets, organisations, and repeated captures. Such tests should mask host identities to prevent apparent relational gains from reflecting machines historically associated with alerts. Learned relation weights may aid diagnosis, but causal interpretation would still require relation ablation and counterfactual removal of context.

7.3.3 Streaming Deployment, Efficiency, and Robustness

Although the formulation is compatible with online inference, streaming behaviour remains untested. A deployment implementation must maintain bounded buffers for each relation, evict expired entries, process out-of-order records, and define behaviour under packet loss, timestamp noise, missing features, NAT, and highly shared servers. Evaluation should span PCAP processing, Suricata extraction, both learning stages, and alert generation. Alongside predictive metrics, realistic-load tests should report throughput, peak memory, and median, 95th-percentile, and 99th-percentile latency.

Bibliography

- [1] Martin Roesch. Snort: Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA)*, 1999.
- [2] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy*, pages 305–316, 2010. doi:10.1109/SP.2010.25.
- [3] Adit Sharma and Arash Habibi Lashkari. A survey on encrypted network traffic: A comprehensive survey of identification/classification techniques, challenges, and future directions. *Computer Networks*, 257:110984, 2025. URL: <https://www.sciencedirect.com/science/article/pii/S1389128624008168>, doi:10.1016/j.comnet.2024.110984.
- [4] Wenqi Dong, Jing Yu, Xinjie Lin, Kewei Chen, Qianwei Meng, Qingjun Yuan, and Yongjuan Wang. Deep learning and pre-training technology for encrypted traffic classification: A comprehensive review. *Neurocomputing*, 617:128444, 2025. doi:10.1016/j.neucom.2024.128444.
- [5] Open Information Security Foundation. Suricata user guide. <https://docs.suricata.io/>, 2026.
- [6] Ali Hozouri, Abbas Mirzaei, and Mehdi Effatparvar. A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges. *Discover Applied Sciences*, 2025. doi:10.1007/s44163-025-00578-1.
- [7] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016. doi:10.1109/COMST.2015.2494502.
- [8] Mohammad Ahmed, Abdun Naser Mahmood, and Jiankun Hu. A survey of network anomaly detection techniques. *Journal of Network and Computer Applications*, 60:19–31, 2016. doi:10.1016/j.jnca.2015.11.016.
- [9] Chuanlong Yin, Yuefei Zhu, Jinlong Fei, and Xinzheng He. A deep learning approach for intrusion detection using recurrent neural networks. *IEEE Access*, 5:21954–21961, 2017. doi:10.1109/ACCESS.2017.2762418.
- [10] Nathan Shone, Tran Ngoc N. Ngoc, Vu Dinh Phai, and Qi Shi. A deep learning approach to network intrusion detection. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2(1):41–50, 2018. doi:10.1109/TETCI.2017.2772792.
- [11] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018. doi:10.14722/ndss.2018.23204.

- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [13] Zihan Wu, Hong Zhang, Peng Wang, and Zhen Sun. Rtids: A robust transformer-based approach for intrusion detection system. *IEEE Access*, 10:64375–64387, 2022. doi:[10.1109/ACCESS.2022.3182333](https://doi.org/10.1109/ACCESS.2022.3182333).
- [14] Xueying Han, Susu Cui, Song Liu, Chen Zhang, Bo Jiang, and Zhigang Lu. Network intrusion detection based on n-gram frequency and time-aware transformer. *Computers & Security*, 128:103171, 2023.
- [15] Liam Daly Manocchio, Siamak Layeghy, Wai Weng Lo, Gayan K Kulatilleke, Mohanad Sarhan, and Marius Portmann. Flowtransformer: A transformer framework for flow-based network intrusion detection systems. *Expert Systems with Applications*, 241:122564, 2024.
- [16] Shadi Aljawarneh, Muneer Aldwairi, and Muneer Yassein. Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model. *Journal of Computational Science*, 25:152–160, 2018. doi:[10.1016/j.jocs.2017.03.006](https://doi.org/10.1016/j.jocs.2017.03.006).
- [17] Farhan Ullah, Shamsheer Ullah, Gautam Srivastava, and Jerry Chun-Wei Lin. IDS-INT: Intrusion detection system using transformer-based transfer learning for imbalanced network traffic. *Digital Communications and Networks*, 10(1):190–204, 2024. doi:[10.1016/j.dcan.2023.03.008](https://doi.org/10.1016/j.dcan.2023.03.008).
- [18] Haoyu Liu and Paul Patras. NetSentry: A deep learning approach to detecting incipient large-scale network attacks. *Computer Communications*, 191:119–132, 2022. doi:[10.1016/j.comcom.2022.04.020](https://doi.org/10.1016/j.comcom.2022.04.020).
- [19] Zhenyue Long, Huiru Yan, Guiquan Shen, Xiaolu Zhang, Haoyang He, and Long Cheng. A transformer-based network intrusion detection approach for cloud security. *Journal of Cloud Computing*, 13(1):5, 2024.
- [20] Xuefeng Jiang, Liuquan Xu, and Li Yu. MFT: A novel memory flow transformer efficient intrusion detection method. *Computers & Security*, 148:104174, 2025. doi:[10.1016/j.cose.2024.104174](https://doi.org/10.1016/j.cose.2024.104174).
- [21] Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, and Jing Yu. Etb-ert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM Web Conference*, pages 633–642, 2022.
- [22] Ruijie Zhao, Mingwei Zhan, Xianwen Deng, Fangqi Li, Yanhao Wang, Yijun Wang, Guan Gui, and Zhi Xue. A novel self-supervised framework based on masked autoencoder for traffic classification. *IEEE/ACM Transactions on Networking*, 32(3):2012–2025, 2024.
- [23] Ke Xu, Xixi Zhang, Yu Wang, Tomoaki Ohtsuki, Bamidele Adebisi, Hikmet Sari, and Guan Gui. Self-supervised learning malware traffic classification based on masked autoencoder. *IEEE Internet of Things Journal*, 11(10):17330–17340, 2024. doi:[10.1109/JIOT.2024.3357072](https://doi.org/10.1109/JIOT.2024.3357072).

- [24] Longlong Li, Yuliang Lu, Guozheng Yang, and Xuehu Yan. End-to-end network intrusion detection based on contrastive learning. *Sensors*, 24(7):2122, 2024. doi:[10.3390/s24072122](https://doi.org/10.3390/s24072122).
- [25] Hong Huang, Ye Lu, Shaohua Zhou, Xingxing Zhang, and Ze Li. CoTNeT: Contextual transformer network for encrypted traffic classification. *Egyptian Informatics Journal*, 26:100475, 2024. doi:[10.1016/j.eij.2024.100475](https://doi.org/10.1016/j.eij.2024.100475).
- [26] Qingyang Li, Yihang Zhang, Zhidong Jia, Yannan Hu, Lei Zhang, Jianrong Zhang, Yongming Xu, Yong Cui, Zongming Guo, and Xinggong Zhang. Dollm: How large language models understanding network flow data to detect carpet bombing ddos. *arXiv preprint arXiv:2405.07638*, 2024.
- [27] Xiaoheng Deng, Jincai Zhu, Xinjun Pei, Lan Zhang, Zhen Ling, and Kaiping Xue. Flow topology-based graph convolutional network for intrusion detection in label-limited iot networks. *IEEE Transactions on Network and Service Management*, 20(1):684–696, 2022.
- [28] Renjie Xu, Guangwei Wu, Weiping Wang, Xing Gao, An He, and Zhengpeng Zhang. Applying self-supervised learning to network intrusion detection for network flows with graph neural network. *arXiv preprint arXiv:2403.01501*, 2024.
- [29] Meihui Zhong, Mingwei Lin, Chao Zhang, and Zeshui Xu. A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges. *Computers & Security*, 141:103821, 2024.
- [30] Sergio Martin-Reizabal, Adrian Caballero-Quiroga, Beatriz Gil-Arroyo, Nuño Basurto, and Ruben Ruiz-Gonzalez. Transformer-based classification of IoT network traffic with flow-to-window aggregation. *Internet of Things*, 36:101879, 2026. doi:[10.1016/j.iot.2026.101879](https://doi.org/10.1016/j.iot.2026.101879).
- [31] Abdelkader El Mahdaouy, Issam Ait Yahia, Soufiane Oualil, and Ismail Berrada. Deep learning for contextualized NetFlow-based network intrusion detection: Methods, data, evaluation and deployment. *arXiv preprint arXiv:2602.05594*, 2026. doi:[10.48550/arXiv.2602.05594](https://doi.org/10.48550/arXiv.2602.05594).
- [32] Ali Shiravi, Hadi Shiravi, Mahbod Tavallaei, and Ali A. Ghorbani. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Computers & Security*, 31(3):357–374, 2012. doi:[10.1016/j.cose.2011.12.012](https://doi.org/10.1016/j.cose.2011.12.012).
- [33] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. A survey of network-based intrusion detection data sets. *Computers & Security*, 86:147–167, 2019. doi:[10.1016/j.cose.2019.06.005](https://doi.org/10.1016/j.cose.2019.06.005).
- [34] Patrik Goldschmidt and Daniela Chudá. Network intrusion datasets: A survey, limitations, and recommendations. *Computers & Security*, 2025.
- [35] Daniela Pinto, Ivone Amorim, Eva Maia, and Isabel Praça. A review on intrusion detection datasets: Tools, processes, and features. *Computer Networks*, 262:111177, 2025. doi:[10.1016/j.comnet.2025.111177](https://doi.org/10.1016/j.comnet.2025.111177).
- [36] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116. SciTePress, 2018. doi:[10.5220/0006639801080116](https://doi.org/10.5220/0006639801080116).

- [37] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282, 2017. doi:[10.14778/3157794.3157797](https://doi.org/10.14778/3157794.3157797).
- [38] Curtis G. Northcutt, Lu Jiang, and Isaac L. Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021. doi:[10.1613/JAIR.1.12125](https://doi.org/10.1613/JAIR.1.12125).
- [39] Takaya Saito and Marc Rehmsmeier. The precision–recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. doi:[10.1371/journal.pone.0118432](https://doi.org/10.1371/journal.pone.0118432).
- [40] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don’ts of machine learning in computer security. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3971–3988. USENIX Association, 2022. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>.
- [41] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018. doi:[10.1609/aaai.v32i1.11671](https://doi.org/10.1609/aaai.v32i1.11671).
- [42] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.
- [43] Yoon Kim. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1746–1751. Association for Computational Linguistics, 2014. doi:[10.3115/v1/D14-1181](https://doi.org/10.3115/v1/D14-1181).
- [44] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi:[10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [45] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734. Association for Computational Linguistics, 2014. doi:[10.3115/v1/D14-1179](https://doi.org/10.3115/v1/D14-1179).
- [46] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations (ICLR)*, 2019. URL: <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [47] Nils Reimers and Iryna Gurevych. Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 338–348, Copenhagen, Denmark, 2017. Association for Computational Linguistics. URL: <https://aclanthology.org/D17-1035/>, doi:[10.18653/v1/D17-1035](https://doi.org/10.18653/v1/D17-1035).

- [48] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. *Proceedings of Machine Learning and Systems*, 3, 2021. URL: <https://proceedings.mlsys.org/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html>.
- [49] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL: <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [50] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. doi:10.1016/j.patrec.2005.10.010.
- [51] Google. Google colab: Frequently asked questions. Google Research, 2026. Accessed 14 July 2026. URL: <https://research.google.com/colaboratory/faq.html>.
- [52] NVIDIA Corporation. NVIDIA L4 GPU Accelerator product brief. Technical Report PB-11316-001-v01, NVIDIA Corporation, 2023. URL: https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/14/PB-11316-001_v01.pdf.

A Complete Packet and Flow Field Specification

Tables A.1 and A.2 document the complete raw schemas emitted by the custom Suricata output module. The extractor calculates means, standard deviations, variances, minima, and maxima online. Sample definitions are used for standard deviations and variances when at least two observations are available. The values described below precede the train-fitted transformations used by Stage 1.

TABLE A.1: Complete packet CSV field specification.

Field	Implementation meaning and unit before transformation
<code>record_type</code>	Constant logging descriptor packet ; no physical unit.
<code>packet_id</code>	Packet counter supplied by Suricata for the replayed capture; integer identifier with no physical unit.
<code>timestamp_us</code>	Packet timestamp expressed as microseconds since the Unix epoch.
<code>ts_sec</code>	Whole-second component of the packet timestamp, in seconds since the Unix epoch.
<code>ts_usec</code>	Microsecond component within <code>ts_sec</code> , in microseconds.
<code>flow_id</code>	Suricata bidirectional-flow identifier used to align packet and flow records; integer identifier with no physical unit.
<code>direction</code>	Suricata flow direction: 0 for to-server, 1 for to-client, and -1 when direction is unavailable.
<code>ip_version</code>	IP version: normally 4 for IPv4, 6 for IPv6, and 0 when unavailable.
<code>src_ip</code>	Source IPv4 or IPv6 address as a string.
<code>dst_ip</code>	Destination IPv4 or IPv6 address as a string.
<code>src_port</code>	TCP or UDP source port number; zero when no supported transport header is available.
<code>dst_port</code>	TCP or UDP destination port number; zero when no supported transport header is available.
<code>protocol</code>	IP protocol number, for example 6 for TCP, 17 for UDP, or 1 for ICMPv4.
<code>pkt_len</code>	Captured packet length returned by Suricata's packet-length accessor, in bytes.
<code>ip_len</code>	IPv4 total length, or the IPv6 40-byte fixed header plus its payload-length field, in bytes; captured packet length is used as a fallback.
<code>payload_len</code>	Suricata packet payload length, in bytes; this is a scalar length and not payload content.
<code>ttl_or_hop_limit</code>	IPv4 time-to-live or IPv6 hop limit, in hops; -1 when unavailable.

Continued on next page

Table A.1 – continued from previous page

Field	Implementation meaning and unit before transformation
has_tcp	One when a TCP header is available; otherwise zero.
has_udp	One when a UDP header is available; otherwise zero.
has_icmp	One when an ICMPv4 header is available; otherwise zero.
tcp_seq	TCP sequence number in host byte order; zero for non-TCP packets.
tcp_ack	TCP acknowledgement number in host byte order; zero for non-TCP packets.
tcp_flags	Integer bit mask containing the TCP control flags; zero for non-TCP packets.
tcp_fin	One when the TCP FIN flag is present; otherwise zero.
tcp_syn	One when the TCP SYN flag is present; otherwise zero.
tcp_rst	One when the TCP RST flag is present; otherwise zero.
tcp_psh	One when the TCP PSH flag is present; otherwise zero.
tcp_ack_flag	One when the TCP ACK flag is present; otherwise zero.
tcp_urg	One when the TCP URG flag is present; otherwise zero.
tcp_ece	One when the TCP ECE flag is present; otherwise zero.
tcp_cwr	One when the TCP CWR flag is present; otherwise zero.
tcp_window	Raw, unscaled 16-bit advertised TCP receive-window value; zero for non-TCP packets.
tcp_header_len	TCP header length, in bytes; zero when no TCP header is available.
udp_len	UDP length field, including the UDP header and data, in bytes; zero for non-UDP packets.
icmp_type	ICMPv4 type; zero when no ICMPv4 header is available.
icmp_code	ICMPv4 code; zero when no ICMPv4 header is available.
l7_proto	Suricata application-layer protocol identifier associated with the flow; zero when unavailable.
flow_iat_us	Elapsed time since the preceding packet in the same bidirectional flow, in microseconds; zero when no valid predecessor exists.
direction_iat_us	Elapsed time since the preceding packet in the same Suricata direction, in microseconds; zero when no valid same-direction predecessor exists.
packet_label	Binary alert-derived target: one when the packet has at least one Suricata alert, otherwise zero.

TABLE A.2: Complete flow CSV field specification.

Field	Implementation meaning and unit before transformation
<code>flow_id</code>	Suricata bidirectional-flow identifier shared with the packet CSV; integer identifier with no physical unit.
<code>flow_start_timestamp_us</code>	Timestamp of the earliest logged packet in the flow, in microseconds since the Unix epoch.
<code>flow_end_timestamp_us</code>	Timestamp of the latest logged packet in the flow, in microseconds since the Unix epoch.
<code>source_ip</code>	Source address recorded for the bidirectional flow, as an IPv4 or IPv6 string.
<code>source_port</code>	Source TCP or UDP port recorded for the flow; zero when unavailable.
<code>destination_ip</code>	Destination address recorded for the bidirectional flow, as an IPv4 or IPv6 string.
<code>destination_port</code>	Destination TCP or UDP port recorded for the flow; zero when unavailable.
<code>protocol</code>	IP protocol number associated with the flow.
<code>flow_duration</code>	Difference between the final and first logged packet timestamps, in microseconds.
<code>total_fwd_packets</code>	Number of to-server packets in the flow.
<code>total_backward_packets</code>	Number of to-client packets in the flow.
<code>total_length_of_fwd_packets</code>	Sum of forward packet payload lengths, in bytes.
<code>total_length_of_bwd_packets</code>	Sum of backward packet payload lengths, in bytes.
<code>fwd_packet_length_max</code>	Maximum forward packet payload length, in bytes.
<code>fwd_packet_length_min</code>	Minimum forward packet payload length, in bytes.
<code>fwd_packet_length_mean</code>	Mean forward packet payload length, in bytes.
<code>fwd_packet_length_std</code>	Sample standard deviation of forward packet payload length, in bytes.
<code>bwd_packet_length_max</code>	Maximum backward packet payload length, in bytes.
<code>bwd_packet_length_min</code>	Minimum backward packet payload length, in bytes.
<code>bwd_packet_length_mean</code>	Mean backward packet payload length, in bytes.
<code>bwd_packet_length_std</code>	Sample standard deviation of backward packet payload length, in bytes.
<code>flow_bytes_per_s</code>	Sum of forward and backward payload bytes divided by flow duration, in bytes per second.
<code>flow_packets_per_s</code>	Total packet count divided by flow duration, in packets per second.
<code>flow_iat_mean</code>	Mean inter-arrival time between successive packets in the bidirectional flow, in microseconds.
<code>flow_iat_std</code>	Sample standard deviation of bidirectional-flow inter-arrival time, in microseconds.
<code>flow_iat_max</code>	Maximum bidirectional-flow inter-arrival time, in microseconds.
<code>flow_iat_min</code>	Minimum bidirectional-flow inter-arrival time, in microseconds.

Continued on next page

Table A.2 – continued from previous page

Field	Implementation meaning and unit before transformation
fwd_iat_total	Elapsed time from the first to the last forward packet, in microseconds.
fwd_iat_mean	Mean inter-arrival time between successive forward packets, in microseconds.
fwd_iat_std	Sample standard deviation of forward inter-arrival time, in microseconds.
fwd_iat_max	Maximum forward inter-arrival time, in microseconds.
fwd_iat_min	Minimum forward inter-arrival time, in microseconds.
bwd_iat_total	Elapsed time from the first to the last backward packet, in microseconds.
bwd_iat_mean	Mean inter-arrival time between successive backward packets, in microseconds.
bwd_iat_std	Sample standard deviation of backward inter-arrival time, in microseconds.
bwd_iat_max	Maximum backward inter-arrival time, in microseconds.
bwd_iat_min	Minimum backward inter-arrival time, in microseconds.
fwd_psh_flags	Number of forward TCP packets carrying the PSH flag.
bwd_psh_flags	Number of backward TCP packets carrying the PSH flag.
fwd_urg_flags	Number of forward TCP packets carrying the URG flag.
bwd_urg_flags	Number of backward TCP packets carrying the URG flag.
fwd_header_length	Sum of forward IP and layer-4 header lengths, in bytes.
bwd_header_length	Sum of backward IP and layer-4 header lengths, in bytes.
fwd_packets_per_s	Forward packet count divided by flow duration, in packets per second.
bwd_packets_per_s	Backward packet count divided by flow duration, in packets per second.
min_packet_length	Minimum payload length over all packets in the flow, in bytes.
max_packet_length	Maximum payload length over all packets in the flow, in bytes.
packet_length_mean	Mean payload length over all packets in the flow, in bytes.
packet_length_std	Sample standard deviation of payload length over all packets, in bytes.
packet_length_variance	Sample variance of payload length over all packets, in squared bytes.
fin_flag_count	Number of TCP packets carrying the FIN flag.
syn_flag_count	Number of TCP packets carrying the SYN flag.

Continued on next page

Table A.2 – continued from previous page

Field	Implementation meaning and unit before transformation
<code>rst_flag_count</code>	Number of TCP packets carrying the RST flag.
<code>psh_flag_count</code>	Number of TCP packets carrying the PSH flag.
<code>ack_flag_count</code>	Number of TCP packets carrying the ACK flag.
<code>urg_flag_count</code>	Number of TCP packets carrying the URG flag.
<code>cwe_flag_count</code>	Legacy CSV field name for the number of TCP packets carrying the CWR flag; the implementation tests <code>TH_CWR</code> .
<code>ece_flag_count</code>	Number of TCP packets carrying the ECE flag.
<code>down_up_ratio</code>	Backward packet count divided by forward packet count; zero when the forward count is zero.
<code>average_packet_size</code>	Total forward and backward payload bytes divided by total packet count, in bytes.
<code>avg_fwd_segment_size</code>	Mean forward payload length, in bytes; equivalent to <code>fwd_packet_length_mean</code> in this extractor.
<code>avg_bwd_segment_size</code>	Mean backward payload length, in bytes; equivalent to <code>bwd_packet_length_mean</code> in this extractor.
<code>has_init_win_bytes_forward</code>	One when an initial forward TCP receive-window value was observed; otherwise zero.
<code>init_win_bytes_forward</code>	Raw, unscaled advertised TCP receive-window value from the first observed forward TCP packet; zero when unavailable.
<code>has_init_win_bytes_backward</code>	One when an initial backward TCP receive-window value was observed; otherwise zero.
<code>init_win_bytes_backward</code>	Raw, unscaled advertised TCP receive-window value from the first observed backward TCP packet; zero when unavailable.
<code>act_data_pkt_fwd</code>	Number of forward TCP packets whose Suricata payload length is greater than zero.
<code>min_seg_size_forward</code>	Minimum layer-4 header length observed in the forward direction, in bytes; this follows the implementation despite the legacy field name.
<code>active_mean</code>	Mean duration of active periods, in microseconds; a new period begins after a flow IAT greater than 5,000,000 microseconds.
<code>active_std</code>	Sample standard deviation of active-period durations, in microseconds.
<code>active_max</code>	Maximum duration of an active period, in microseconds.
<code>active_min</code>	Minimum duration of an active period, in microseconds.
<code>idle_mean</code>	Mean duration of idle gaps exceeding 5,000,000 microseconds, in microseconds.
<code>idle_std</code>	Sample standard deviation of the durations of such idle gaps, in microseconds.

Continued on next page

Table A.2 – continued from previous page

Field	Implementation meaning and unit before transformation
<code>idle_max</code>	Maximum duration of such an idle gap, in microseconds.
<code>idle_min</code>	Minimum duration of such an idle gap, in microseconds.
<code>label</code>	Binary flow target: one when at least one packet in the flow has a Suricata alert, otherwise zero.